

Course Submission Cover Sheet

Module: CC6058 Big Data and Visualisation

Assignment no: 001

Weighting: 100% of module mark

Deadline: 3PM Wednesday the 22nd of January 2025

Module Leader: Sandra Fernando

Student ID: 21022451



PLAGIARISM

Please be aware that there are specific regulations regarding plagiarism and the use of AI. A summary of these regulations is provided below.

This header sheet should be attached to the work you submit.

Academic integrity is fundamental to your academic work and studies. It means being honest in your efforts ensuring that you properly acknowledge and give credit to the work and ideas of others when incorporating them into your arguments. All assessment submissions must be entirely your own work, based on your personal learning and appropriately referenced, including any use of Generative AI tools. Exceptions are made only for tasks where extensive and critical use of a specific AI tool is explicitly part of the assessment brief. The University considers the misuse of Generative AI applications or other form of plagiarism to deceive or gain an unfair advantage as academic misconduct. This includes:

- Using AI tools to generate content or ideas and presenting them without proper citation is considered plagiarism while creating and submitting an entire assignment using Generative AI as your own work is similar to contract cheating, similar to hiring an 'Essay Mill' or third party. <https://student.londonmet.ac.uk/your-studies/student-administration/guidance-on-the-use-of-artificial-intelligence/>
- Academic misconduct also includes falsifying data, having someone else write your assessment on your behalf (personation), bribery, collusion, plagiarism, and other actions aimed at gaining an unfair advantage in assessments. Assisting in any of these acts is also considered misconduct. It is important to note that copying another student's work or failing to properly attribute ideas and extracts from published sources constitutes an

Summary/Abstract

This report consists of two distinct analytical projects. The first part presents an NHS England prescription data analysis, focusing on metformin prescriptions for diabetes management, utilising Apache Spark for big data processing. The analysis involved comprehensive data cleaning, feature engineering, and machine learning techniques. A linear regression model was developed using both original and log-transformed target variables to predict actual prescription costs, with the log-transformed model demonstrating superior performance ($R^2 = 0.995$) compared to the original target variable model ($R^2 = 0.63$). The data visualisation phase, conducted using Power BI, identified Goodman's Field Health Centre as the highest prescriber with 1.72 million prescriptions and August as the peak month for prescription costs at 51.5K. The second part of the study implemented a real-time data streaming system using Apache Kafka to monitor IBM stock prices. This system featured a producer-consumer architecture that successfully captured live stock data from the Alpha Vantage API at 5-minute intervals. It was processed to calculate descriptive statistics, including average volume, maximum close price, and minimum open price, and the data was stored in CSV format for permanent storage. This study demonstrates the effective application of big data technologies in Batch Processing and Real-Time streaming

Contents

Summary/Abstract	1
Methodology	4
PART 1	5
Introduction.....	5
Data Collection/ Overview	7
Dataset Overview	7
Purpose of the Dataset.....	7
Creating a spark session.....	7
Dataset Description	8
Metadata Table	10
Data Cleaning and Imputation	11
Filling Null Values.....	11
Dropping /Deleting Unwanted Columns.....	12
Transforming Categorical Variables to Numerical	14
New Dataframe With Clean and Sorted Data	16
Exploratory Data Analysis	17
Summary Statistics	17
Correlation	18
Feature Engineering	19
Log Transformation for ACTUAL_COST	20
Log Transformation + Scaling for Input Variables.....	20
Scaling QUANTITY, YEAR, and MONTH	22
Clustering Geo-Positions	22
Machine Learning Phase.....	23
Pipeline.....	23
Model Training	24
Model Evaluation	25
Analysis of results	25
Data Visualisation by Power BI	26
Total number of Prescriptions by GP Practice	26
Time series chart for Actual Cost	26

Geospatial data of the total number of prescriptions	27
Monthly Total Cost of Prescription Vs Actual Cost	27
Interactive Dashboard	28
StreamLit Interactive Dashboard	29
Feature 1	29
Feature 2	29
Feature 3	30
PART 2	32
Real-Time Stock Data Streaming Using Kafka	32
Overview	32
Writing Kafka Producer	32
Writing the Kafka Consumer	34
Extending the Consumer to Display Descriptive Statistics	35
Saving file to .csv file	36
PART 3	37
Reflective Report on Part 1 and Part 2	37
Part 1: NHS England Prescription Data Analysis.....	37
Part B: Real-Time Stock Data Streaming Using Kafka.....	37
Conclusion	38
Appendices	39
Appendix 1. (Pipeline 1)	39
Appendix2:(Pipeline 2)	40
Appendix 3: (Streamlit Dashboard)	41
References	43

Methodology

This report utilises NHS England prescription data focusing on metformin prescriptions for diabetes management, employing Apache Spark as the primary data processing framework through PySpark within a Conda environment using Jupyter Notebook. The data processing begins with comprehensive cleaning and preprocessing steps, where 94 null values in geographical data fields (POSTCODE, PRACTICE_LATITUDE, and PRACTICE_LONGITUDE) for Practice F84714 are identified and filled using existing default records. The preprocessing phase includes removing redundant columns such as BNF_CHAPTER_PLUS_CODE and unnecessary identifiers while retaining geographical coordinates for spatial analysis. Feature transformation is implemented by converting temporal data from YEAR_MONTH string format into separate numerical columns, applying StringIndexer to CHEMICAL_SUBSTANCE_BNF_DESCR due to its low cardinality, and implementing One-Hot Encoding for BNF_DESCRIPTION due to high cardinality. The feature engineering process involves applying log transformation to heavily skewed variables (TOTAL_QUANTITY, ITEMS, NIC, ADQUSAGE, ACTUAL_COST), implementing standard scaling for QUANTITY, YEAR, and MONTH variables, and creating geographical clusters (K=4) based on latitude and longitude to represent Tower Hamlets' postal code areas. Statistical analysis is conducted to understand data distributions and relationships between variables. The modelling phase employs two parallel pipelines: one using log-transformed ACTUAL_COST and another using original values, with an 80-20 train-test split for validation. Linear Regression is selected as the primary model due to the continuous nature of the target variable and the need for interpretable results, with model performance evaluated using multiple metrics, including RMSE, MSE, and R² score. The analysis culminates in comprehensive data visualization using Power BI, creating prescription distribution analyses, time series visualizations of actual costs, geospatial representations of prescription patterns, and an interactive dashboard with practice-specific metrics. Each step in this methodology is documented and validated to ensure reproducibility and maintain analytical integrity.

PART 1

Introduction

Apache Spark and Hadoop MapReduce are prominent big data processing frameworks with distinct characteristics that influence their suitability for various applications. While both are designed to handle large-scale data processing, they differ in several key aspects

1. Performance and Computation Model

Spark utilises an in-memory computation model, significantly reducing the time spent on disk I/O operations. This approach enables Spark to process data up to 100 times faster than MapReduce for specific applications (Chambers and Zaharia, 2018). In contrast, MapReduce writes intermediate results to disk after each map and reduces phase, leading to higher latency and slower performance, especially for iterative algorithms.

2. Ease of Use and Flexibility

Spark provides high-level APIs in languages such as Java, Scala, Python, and R, along with interactive shells, which enhance developer productivity. It also supports advanced analytics through built-in libraries for machine learning (MLlib), graph processing (GraphX), and streaming data (Spark Streaming), offering a unified platform for various data processing tasks (Chambers and Zaharia, 2018). MapReduce, however, requires developers to write custom code for each job, typically in Java, which can be more complex and less flexible.

3. Fault Tolerance Mechanisms

Both frameworks are designed to handle failures in a distributed environment. MapReduce achieves fault tolerance through data replication across nodes, ensuring that data is not lost if a node fails. On the other hand, Spark uses a lineage-based approach with Resilient Distributed Datasets (RDDs), allowing it to recompute lost data partitions efficiently without replicating (Chambers and Zaharia, 2018).

4. Data Processing Models

Spark supports batch and real-time stream processing, making it versatile for various applications. Its ability to handle streaming data is particularly beneficial for use cases requiring low-latency processing. MapReduce is primarily designed for batch processing and lacks native support for real-time data streams, limiting its applicability in scenarios that demand immediate data processing.

5. Resource Management and Scheduling

Spark can run on various cluster managers, including Hadoop YARN, Apache Mesos, and its standalone cluster manager, providing flexibility in deployment. It also offers dynamic resource allocation, allowing it to adjust resource usage based on workload demands. MapReduce is tightly integrated with the Hadoop ecosystem and relies on YARN for resource management, which may result in less flexibility than Spark.

6. Scalability and Performance with Large Datasets

Studies have shown that Spark outperforms MapReduce in processing smaller datasets due to its in-memory capabilities. However, as data size increases, the performance gap narrows, and in some cases, MapReduce may perform better due to its optimised disk I/O operations (Ahmed et al., 2020). This indicates that the choice between Spark and MapReduce should consider the specific data size and workload characteristics.

7. Community Support and Ecosystem

Spark has a rapidly growing community and a rich library and tool ecosystem, facilitating continuous improvements and a wide range of applications. Being an older technology, MapReduce has a more mature but less active community, and its ecosystem is more focused on traditional batch-processing tasks.

While Apache Spark and Hadoop MapReduce can process large datasets, Spark's in-memory computation, ease of use, and versatility make it more suitable for applications requiring fast, iterative, and real-time data processing (Refer to Diagram 1). MapReduce remains a reliable choice for straightforward, batch-oriented tasks, particularly when dealing with huge datasets where disk I/O is less of a bottleneck.

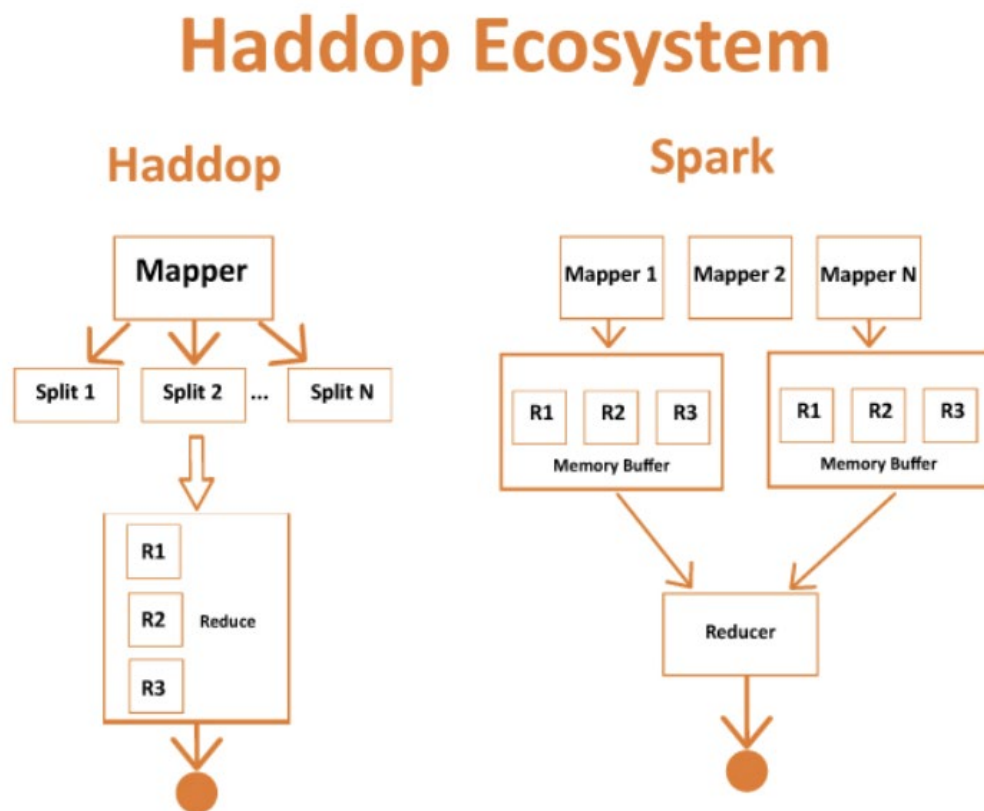


Diagram 1

Data Collection/ Overview

Dataset Overview

This dataset contains prescription data from the NHS in England, focusing on medications and appliances dispensed in the community. Specifically, it includes information about metformin prescriptions for diabetes management and how much it actually costs the NHS. The dataset is structured and contains data related to geographical, organisational, and prescription-level details.

Purpose of the Dataset

The dataset is intended to analyse prescription patterns, costs, and quantities of drugs like metformin across various NHS organisations and practices. It provides insights into regional differences, prescribing behaviour, and financial implications for healthcare management.

The dataset allows for deriving several insightful conclusions. It can help identify metformin prescribing trends, revealing which practices and regions prescribe the medication most frequently. Additionally, it provides an opportunity to analyse monthly prescribing behaviour, highlighting seasonal or time-based variations.

Through cost analysis, the dataset enables calculating the average cost per prescription, considering both the Net Ingredient Cost (NIC) and the adjusted actual cost. This can reveal cost variations across different NHS regions or practices, offering a deeper understanding of resource allocation and potential cost-saving opportunities. For this report, the best prediction model will be implemented to predict the actual cost of the prescription medication (Metformin).

Creating a spark session

For this report, Pyspark will be used to analyse data using the Conda environment and Jupyter Notebook. *Figure 1* below shows how a spark session is created

Key Components in PySpark's Functionality

PySpark, as a library, serves as a Python API to Apache Spark, enabling distributed data processing. Its functionality in a Conda

Apache Spark

The core Spark engine handles distributed computing, typically installed via PySpark's dependencies.

Spark executors and drivers run as JVM processes, managed by PySpark through Python calls.

```
# Initialize Spark session
spark = SparkSession.builder.master("local[1]") \
    .appName("Diabetes") \
    .getOrCreate()

spark
```

SparkSession - in-memory

SparkContext

Spark UI

Version	v3.5.3
Master	local[1]
AppName	Diabetes

Figure 1

Dataset Description

Figure 1.1 below shows the wide range of values in variables such as **TOTAL_QUANTITY** (1–58,688), **ITEMS** (1–741), and **ADQUSAGE** (0.0–19,562.66665). These wide ranges may require normalisation or scaling to ensure that variables with larger values do not disproportionately influence the model during training. Financial metrics like **NIC** (0.02–1760.64) and **ACTUAL_COST** (0.1311–1412.33335) will also need appropriate scaling.

Categorical variables such as **PRACTICE_CODE**, **CHEMICAL_SUBSTANCE_BNF_DESCR**, and **BNF_DESCRIPTION**, with varying cardinalities (e.g., 31 and 4), may require encoding methods like one-hot encoding or target encoding, depending on the model's sensitivity to dimensionality. The single cardinality of **BNF_CHAPTER_PLUS_CODE** may not provide significant variation for predictive modelling and could be excluded or considered a constant feature.

Geographic features (**PRACTICE_LATITUDE** and **PRACTICE_LONGITUDE**) can add spatial insights but might require transformation into distance metrics or clustering to capture meaningful relationships.

Variable	Data Type	Range	Cardinality
TOTAL_QUANTITY	int	(1, 58688)	N/A
PRACTICE_CODE	string	N/A	31
ITEMS	int	(1, 741)	N/A
CHEMICAL_SUBSTANCE_BNF_DESCR	string	N/A	4
BNF_CHAPTER_PLUS_CODE	string	N/A	1
POSTCODE	string	N/A	31
YEAR_MONTH	string	N/A	13
NIC	double	(0.02, 1760.64)	N/A
ADQUSAGE	double	(0.0, 19562.66665)	N/A
PRACTICE_NAME	string	N/A	31
ACTUAL_COST	double	(0.1311, 1412.33335)	N/A
BNF_DESCRIPTION	string	N/A	28
QUANTITY	int	(1, 1200)	N/A
PRACTICE_LATITUDE	double	(51.4911493, 51.5363018)	N/A
PRACTICE_LONGITUDE	double	(-0.0741499, -0.004900067)	N/A

Figure 1.1

```

from pyspark.sql.functions import col, countDistinct, min, max
from tabulate import tabulate

variable_summary = []

# List of all columns in the dataset
columns = data.columns

for column in columns:
    data_type = dict(data.dtypes).get(column)
    summary = {"Variable": column, "Data Type": data_type}

    if data_type in ["int", "double"]: # Numerical columns
        # Calculate range (min and max)
        min_val = data.select(min(col(column))).collect()[0][0]
        max_val = data.select(max(col(column))).collect()[0][0]
        summary["Range"] = f"({min_val}, {max_val})"
        summary["Cardinality"] = "N/A" # Not applicable for numerical columns

    elif data_type == "string": # Categorical columns
        # Calculate cardinality (number of unique values)
        cardinality = data.select(countDistinct(col(column))).collect()[0][0]
        summary["Range"] = "N/A" # Not applicable for categorical columns
        summary["Cardinality"] = cardinality

    variable_summary.append(summary)

# Convert dictionaries to a list of rows for tabulate
table_data = [[item["Variable"], item["Data Type"], item["Range"], item["Cardinality"]] for item in variable_summary]
headers = ["Variable", "Data Type", "Range", "Cardinality"]

# Print the table
print(tabulate(table_data, headers=headers, tablefmt="grid"))

```

Screenshot 1.1

Metadata Table

Figure 2 below shows the metadata table for the Diabetes dataset.

```
root
|-- TOTAL_QUANTITY: integer (nullable = true)
|-- PRACTICE_CODE: string (nullable = true)
|-- ITEMS: integer (nullable = true)
|-- CHEMICAL_SUBSTANCE_BNF_DESCR: string (nullable = true)
|-- BNF_CHAPTER_PLUS_CODE: string (nullable = true)
|-- POSTCODE: string (nullable = true)
|-- YEAR_MONTH: string (nullable = true)
|-- NIC: double (nullable = true)
|-- ADQUSAGE: double (nullable = true)
|-- PRACTICE_NAME: string (nullable = true)
|-- ACTUAL_COST: double (nullable = true)
|-- BNF_DESCRIPTION: string (nullable = true)
|-- QUANTITY: integer (nullable = true)
|-- PRACTICE_LATITUDE: double (nullable = true)
|-- PRACTICE_LONGITUDE: double (nullable = true)

Total number of rows: 14078
Total number of columns: 15
Unique value counts for each column:
Column 'TOTAL_QUANTITY' has 736 unique values.
Column 'PRACTICE_CODE' has 31 unique values.
Column 'ITEMS' has 357 unique values.
Column 'CHEMICAL_SUBSTANCE_BNF_DESCR' has 4 unique values.
Column 'BNF_CHAPTER_PLUS_CODE' has 1 unique values.
Column 'POSTCODE' has 32 unique values.
Column 'YEAR_MONTH' has 13 unique values.
Column 'NIC' has 2872 unique values.
Column 'ADQUSAGE' has 822 unique values.
Column 'PRACTICE_NAME' has 31 unique values.
Column 'ACTUAL_COST' has 8528 unique values.
Column 'BNF_DESCRIPTION' has 28 unique values.
Column 'QUANTITY' has 100 unique values.
Column 'PRACTICE_LATITUDE' has 32 unique values.
Column 'PRACTICE_LONGITUDE' has 32 unique values.
```

Figure 2

```
# Read the CSV file into a DataFrame
data = spark.read.csv("C:/Users/showg/OneDrive/Desktop/Big Data/Diabetes.csv", header=True, inferSchema=True)

# Display schema and data
data.printSchema()

# Total number of rows
total_rows = data.count()
print(f"Total number of rows: {total_rows}")

# Total number of columns
total_columns = len(data.columns)
print(f"Total number of columns: {total_columns}")

# Unique counts for each column
print("Unique value counts for each column:")
for column in data.columns:
    unique_count = data.select(column).distinct().count()
    print(f"Column '{column}' has {unique_count} unique values.")
```

Screenshot 2

From the above metadata table (*Figure 2*), the following variables appear to be **categorical** based on their data type and unique counts:

PRACTICE_CODE

- Represents unique identifiers for practices (categorical codes).
- 31 unique codes.

CHEMICAL_SUBSTANCE_BNF_DESCR:

- Describes the chemical substance (e.g., Metformin hydrochloride).
- 4 unique descriptions.

BNF_CHAPTER_PLUS_CODE:

- Categorical classification based on the British National Formulary (e.g., 06: Endocrine System).
- 1 unique category in this dataset (indicating a single BNF chapter).

POSTCODE:

- Represents locations or areas (categorical by region).
- 32 unique postcodes.

YEAR_MONTH:

- Variable type needs to change from string to Data

PRACTICE_NAME:

- Represents the name of the medical practices (e.g., THE JUBILEE STREET PRACTICE LTD).
- 28 unique names.

BNF_DESCRIPTION:

- Categorical descriptions of specific drugs (e.g., Metformin 500mg tablets).
- 128 unique descriptions.

Most columns have non-null values apart from the columns **POSTCODE**, **PRACTICE_LATITUDE** and **PRACTICE_LONGITUDE**, each variable missing 94 values. This will be addressed later with the data imputation method.

The **BNF_CHAPTER_PLUS_CODE** variable contains only one unique categorical feature; this column will be dropped as it has zero variance in statistical terms, making it redundant for analysis and machine learning.

Data Cleaning and Imputation

Filling Null Values

Figure 3 shows the missing 94 null values for the Practice with the code F84714 for Columns **POSTCODE**, **PRACTICE_LATITUDE**, and **PRACTICE_LONGITUDE**. As the dataset already contains the following information for Practice F8714, the null values will be replaced with the default records.

PRACTICE_CODE	ITEMS	CHEMICAL_BNF_CHAPTER_PLUS_CODE	POSTCODE	YEAR	MO	NIC	ADQSAG_PRACTICE_NAME	ACTUAL_CBNF_DESC	QUANTITY	PRACTICE_LATITUDE	PRACTICE_LONGITUDE
F84714	50	Metformin 06: Endocrine System		23-Oct	9	116.6667	ST.PAUL'S WAY MEDICAL CTR	14.05041	Metformin	7	
F84714	2	Metformin 06: Endocrine System		23-Oct	4.86	56	ST.PAUL'S WAY MEDICAL CTR	4.77722	Metformin	84	
F84714	6	Metformin 06: Endocrine System		23-Oct	12.78	168	ST.PAUL'S WAY MEDICAL CTR	12.04558	Metformin	84	
F84714	398	Metformin 06: Endocrine System		23-Oct	282.58	3714.667	ST.PAUL'S WAY MEDICAL CTR	269.6314	Metformin	28	
F84714	1	Metformin 06: Endocrine System		23-Oct	2.06	63.46667	ST.PAUL'S WAY MEDICAL CTR	1.94203	Metformin	112	
F84714	5	Metformin 06: Endocrine System		23-Oct	3.15	46.66667	ST.PAUL'S WAY MEDICAL CTR	3.51264	Metformin	14	
F84714	1	Metformin 06: Endocrine System		23-Oct	15.88	149.3333	ST.PAUL'S WAY MEDICAL CTR	14.88739	Sukkarto S	224	
F84714	25	Metformin 06: Endocrine System		23-Oct	162	1866.667	ST.PAUL'S WAY MEDICAL CTR	152.0574	Metformin	224	
F84714	34	Metformin 06: Endocrine System		23-Oct	48.28	634.6667	ST.PAUL'S WAY MEDICAL CTR	45.64607	Metformin	56	
F84714	91	Metformin 06: Endocrine System		23-Oct	258.44	3397.333	ST.PAUL'S WAY MEDICAL CTR	243.2123	Metformin	112	
F84714	1	Metformin 06: Endocrine System		23-Oct	3.97	37.33333	ST.PAUL'S WAY MEDICAL CTR	3.73115	Sukkarto S	56	
F84714	12	Metformin 06: Endocrine System		23-Oct	6.36	84	ST.PAUL'S WAY MEDICAL CTR	7.30629	Metformin	21	
F84714	6	Metformin 06: Endocrine System		23-Oct	59.52	448	ST.PAUL'S WAY MEDICAL CTR	55.82751	Sukkarto S	224	
F84714	1	Metformin 06: Endocrine System		23-Oct	5.06	74.66667	ST.PAUL'S WAY MEDICAL CTR	4.75216	Metformin	112	
F84714	1	Metformin 06: Endocrine System		23-Oct	2.53	37.33333	ST.PAUL'S WAY MEDICAL CTR	2.38228	Metformin	56	
F84714	1	Metformin 06: Endocrine System		23-Oct	9.72	112	ST.PAUL'S WAY MEDICAL CTR	9.11724	Metformin	336	
F84714	1	Metformin 06: Endocrine System		23-Oct	33.26	28	ST.PAUL'S WAY MEDICAL CTR	31.16745	Metformin	56	
F84714	21	Metformin 06: Endocrine System		23-Oct	68.04	784	ST.PAUL'S WAY MEDICAL CTR	63.9943	Metformin	112	
F84714	25	Metformin 06: Endocrine System		23-Oct	10.25	116.6667	ST.PAUL'S WAY MEDICAL CTR	12.4113	Metformin	14	
F84714	7	Metformin 06: Endocrine System		23-Oct	59.64	784	ST.PAUL'S WAY MEDICAL CTR	55.95232	Metformin	336	
F84714	8	Metformin 06: Endocrine System		23-Oct	34.08	448	ST.PAUL'S WAY MEDICAL CTR	32.02235	Metformin	168	
F84714	1	Metformin 06: Endocrine System		23-Oct	2.48	18.66667	ST.PAUL'S WAY MEDICAL CTR	2.33545	Sukkarto S	56	
F84714	1	Metformin 06: Endocrine System		23-Oct	2.39	18.66667	ST.PAUL'S WAY MEDICAL CTR	2.25114	Yaltormin	56	
F84714	86	Metformin 06: Endocrine System		23-Oct	30.96	401.3333	ST.PAUL'S WAY MEDICAL CTR	38.66701	Metformin	14	

Figure 3

```

file_path = "C:/Users/showg/OneDrive/Desktop/Big Data/Diabetes.csv"
df = spark.read.csv(file_path, header=True, inferSchema=True)

# default values for the columns where PRACTICE_CODE = 'F84714'
default_values = df.filter(col("PRACTICE_CODE") == "F84714") \
    .select("POSTCODE", "PRACTICE_LATITUDE", "PRACTICE_LONGITUDE") \
    .first()

# Extract default values
if default_values:
    default_postcode = default_values["POSTCODE"]
    default_latitude = default_values["PRACTICE_LATITUDE"]
    default_longitude = default_values["PRACTICE_LONGITUDE"]
else:
    raise ValueError("No rows found with PRACTICE_CODE = 'F84714'")

# Replace null values in the respective columns with the default values
df = df.fillna({
    "POSTCODE": default_postcode,
    "PRACTICE_LATITUDE": default_latitude,
    "PRACTICE_LONGITUDE": default_longitude
})

# Filter rows where changes were made (nulls replaced with default values)
df_changes = df

# Display the changed rows
df_changes.show()

```

Screenshot 3

Dropping /Deleting Unwanted Columns

Column Name: **BNF_CHAPTER_PLUS_CODE**

Justification: Contains only one value (06: Endocrine System)

Column Name: **PRACTICE_CODE**

Justification: Not required for this Analysis

Column Name: **PRACTICE_NAME**

Justification: Not required for this Analysis

Column Name: **POSTCODE**

Justification: Longitude and Latitude will be used as they are already in numerical format. *Figure 4* below shows the remaining columns after deletion.

-----+											
TOTAL_QUANTITY	ITEMS	CHEMICAL_SUBSTANCE_BNF_DESCR	YEAR_MONTH	NIC	ADQUASAGE	ACTUAL_COST	BNF_DESCRIPTION	QUANTITY	PRACTICE_LATITUDE	PRACTICE_LONGITUDE	
-----+											
0.05055	15680	70	Metformin hydroch...	23-Jul	442.4	5226.66666	415.30817	Metformin 500mg t...	224	51.5135999	-
830635	112	1	Metformin hydroch...	23-Jul	3.84	37.33333	3.70971	Glucophage 500mg ...	112	51.51835505	-0.03
021075	5152	23	Metformin hydroch...	23-Jul	145.36	1717.33333	136.4584	Metformin 500mg t...	224	51.5363018	-0.027
014907	1064	76	Metformin hydroch...	23-Jul	30.4	354.66667	37.02111	Metformin 500mg t...	14	51.5238708	-0.
0.03887	56	1	Metformin hydroch...	23-Jul	1.66	18.66667	1.56749	Metformin 500mg m...	56	51.52979	-
143071	60	1	Metformin hydroch...	23-Jul	1.69	20.0	1.69559	Metformin 500mg t...	60	51.5141762	-0.0
143071	252	1	Metformin hydroch...	23-Jul	7.11	84.0	6.67305	Metformin 500mg t...	252	51.5141762	-0.0
222367	252	12	Metformin hydroch...	23-Jul	7.08	84.0	7.98134	Metformin 500mg t...	21	51.50533401	-0.058
550705	112	1	Metformin hydroch...	23-Jul	3.32	37.33333	3.12257	Metformin 500mg m...	112	51.5139726	-0.0
266905	504	24	Metformin hydroch...	23-Jul	14.16	168.0	15.96268	Metformin 500mg t...	21	51.52468015	-0.04
696235	224	8	Metformin hydroch...	23-Jul	9.92	74.66667	10.19225	Sukkarto SR 500mg...	28	51.5173976	-0.0
651999	140	4	Metformin hydroch...	23-Jul	3.96	46.66667	4.15933	Metformin 500mg t...	35	51.52377	-0.0
0.06426	252	18	Metformin hydroch...	23-Jul	10.08	168.0	11.46614	Metformin 1g modi...	14	51.5269199	-
0.06426	1512	9	Metformin hydroch...	23-Jul	42.66	504.0	40.07547	Metformin 500mg t...	168	51.5269199	-
014907	84	12	Metformin hydroch...	23-Jul	2.52	28.0	3.70954	Metformin 500mg m...	7	51.5238708	-0.
501745	504	6	Metformin hydroch...	23-Jul	11.28	285.6	11.2415	Metformin 850mg t...	84	51.52589465	-0.02
900067	336	2	Metformin hydroch...	23-Jul	23.82	224.0	22.33937	Sukkarto SR 1000m...	168	51.51448982	-0.004
713573	2184	13	Metformin hydroch...	23-Jul	61.62	728.0	57.88679	Metformin 500mg t...	168	51.50071373	-0.025
0.06426	896	4	Metformin hydroch...	23-Jul	26.56	298.66667	24.931	Metformin 500mg m...	224	51.5135999	-

Figure 4

```
# Drop the specified columns
columns_to_drop = ["POSTCODE", "BNF_CHAPTER_PLUS_CODE", "PRACTICE_CODE", "PRACTICE_NAME"]
df_updated = df_changes.drop(*columns_to_drop)

df_updated.show()
```

Screenshot 4

Transforming Categorical Variables to Numerical

Most machine learning algorithms can only work with numerical inputs since they rely on mathematical operations and calculations. The following Columns below will go through the transformation

Column Name: **YEAR_MONTH**

Type of Encoding: Month mapping

Justification: As this column is labelled as a string, the year and month columns are separated, giving the month column a numerical value rather than a text. *Figure 5* below shows the two new columns

TOTAL_QUANTITY	ITEMS	CHEMICAL_SUBSTANCE_BNF_DESCR	NIC	ADQUSAGE	ACTUAL_COST	BNF_DESCRIPTION
112	8	Metformin hydrochloride	12.8	56.0	12.89024	Metformin 750mg modified-release tablets
-0.02460595	2023 7					
336	3	Metformin hydrochloride	23.82	224.0	22.35177	Sukkarto SR 1000mg tablets
-0.02460595	2023 7					
56	1	Metformin hydrochloride	2.39	18.66667	2.25135	Yaltormin SR 500mg tablets
-0.02460595	2023 7					
56	1	Metformin hydrochloride	2.48	18.66667	2.33566	Sukkarto SR 500mg tablets
-0.02460595	2023 7					
168	1	Metformin hydrochloride	4.98	56.0	4.67766	Metformin 500mg modified-release tablets
-0.02460595	2023 7					
560	20	Metformin hydrochloride	24.8	186.66667	25.48063	Sukkarto SR 500mg tablets

```
from pyspark.sql.functions import col, when, lit, upper

# Convert YEAR_MONTH to upercase to standardize abbreviations
df_updated = df_updated.withColumn("YEAR_MONTH", upper(col("YEAR_MONTH")))

# Extract YEAR as a numeric value
df_updated = df_updated.withColumn("YEAR", (col("YEAR_MONTH").substr(1, 2).cast("int") + 2000))

# Initialize MONTH column with null values
df_updated = df_updated.withColumn("MONTH", lit(None).cast("int"))

# Map MONTH abbreviations to numeric values
month_mapping = {
    "JAN": 1, "FEB": 2, "MAR": 3, "APR": 4, "MAY": 5, "JUN": 6,
    "JUL": 7, "AUG": 8, "SEP": 9, "OCT": 10, "NOV": 11, "DEC": 12
}

# Update MONTH column iteratively
for month, num in month_mapping.items():
    df_updated = df_updated.withColumn(
        "MONTH", when(col("YEAR_MONTH").like(f"%{month}"), lit(num)).otherwise(col("MONTH"))
    )

# Drop the original YEAR_MONTH column
df_updated = df_updated.drop("YEAR_MONTH")

# Display the updated DataFrame
df_updated.show(truncate=False)
```

Screenshot 5

Justification: This method maps categorical string values to numeric indices, which is efficient and works well with most ML algorithms and Low cardinal data. The variable has only two unique data, so the string indexer is suitable for this conversion. *Figure 6* below is the evidence

Figure 6

Screenshot 6

Justification: algorithms like Linear Regression perform better with one-hot encoding for categorical variables with higher cardinality. *Figure 7* below shows the changes

BNF_DESCRIPTION	BNF_DESCRIPTION_INDEX	BNF_DESCRIPTION_OHE
Metformin 750mg modified-release tablets	8.0	(10,[8],[1.0])
Sukkarto SR 1000mg tablets	3.0	(10,[3],[1.0])
Yaltormin SR 500mg tablets	9.0	(10,[9],[1.0])
Sukkarto SR 500mg tablets	2.0	(10,[2],[1.0])
Metformin 500mg modified-release tablets	1.0	(10,[1],[1.0])
Sukkarto SR 500mg tablets	2.0	(10,[2],[1.0])
Metformin 500mg tablets	0.0	(10,[0],[1.0])
Metformin 500mg modified-release tablets	1.0	(10,[1],[1.0])
Sukkarto SR 500mg tablets	2.0	(10,[2],[1.0])
Metformin 500mg tablets	0.0	(10,[0],[1.0])
Metformin 500mg/5ml oral solution sugar free	6.0	(10,[6],[1.0])
Metformin 500mg modified-release tablets	1.0	(10,[1],[1.0])
Metformin 850mg tablets	4.0	(10,[4],[1.0])
Metformin 1g modified-release tablets	5.0	(10,[5],[1.0])
Metformin 500mg tablets	0.0	(10,[0],[1.0])
Metformin 500mg tablets	0.0	(10,[0],[1.0])
Metformin 1g modified-release tablets	5.0	(10,[5],[1.0])
Metformin 500mg modified-release tablets	1.0	(10,[1],[1.0])
Metformin 500mg modified-release tablets	1.0	(10,[1],[1.0])
Metformin 500mg tablets	0.0	(10,[0],[1.0])

only showing top 20 rows

Figure 7

```
from pyspark.ml.feature import StringIndexer, OneHotEncoder

#Index the BNF_DESCRIPTION column (required before OneHotEncoding)
indexer = StringIndexer(inputCol="BNF_DESCRIPTION", outputCol="BNF_DESCRIPTION_INDEX")
df_indexed = indexer.fit(df_updated).transform(df_updated)

#Apply OneHotEncoder to the indexed column
encoder = OneHotEncoder(inputCol="BNF_DESCRIPTION_INDEX", outputCol="BNF_DESCRIPTION_OHE")
df_encoded = encoder.fit(df_indexed).transform(df_indexed)

# Display the updated DataFrame with the new column
df_encoded.select("BNF_DESCRIPTION", "BNF_DESCRIPTION_INDEX", "BNF_DESCRIPTION_OHE").show(truncate=False)
```

Screenshot 7

New Dataframe With Clean and Sorted Data

Figure 8 below shows a data frame with no null values, and all categorical features turned into a numerical value

TOTAL_QUANTITY DESCR_INDEX	ITEMS	NIC	ADQUSAGE	ACTUAL_COST	QUANTITY	PRACTICE_LATITUDE	PRACTICE_LONGITUDE	YEAR	MONTH	BNF_DESCRIPTION_OHE	CHEMICAL_SUBSTANCE_BNF_
112	8	12.8	56.0	12.89024	14	51.51800955	-0.02460595	2023	7	(10,[8],[1.0])	0.0
336	3	23.82	224.0	22.35177	112	51.51800955	-0.02460595	2023	7	(10,[3],[1.0])	0.0
56	1	2.39	18.66667	2.25135	56	51.51800955	-0.02460595	2023	7	(10,[9],[1.0])	0.0
56	1	2.48	18.66667	2.33566	56	51.51800955	-0.02460595	2023	7	(10,[2],[1.0])	0.0
168	1	4.98	56.0	4.67766	168	51.51800955	-0.02460595	2023	7	(10,[1],[1.0])	0.0
560	20	24.8	186.66667	25.48063	28	51.51800955	-0.02460595	2023	7	(10,[2],[1.0])	0.0
1050	75	30.0	350.0	36.53399	14	51.51800955	-0.02460595	2023	7	(10,[0],[1.0])	0.0
2800	25	83.0	933.33333	78.06437	112	51.51800955	-0.02460595	2023	7	(10,[1],[1.0])	0.0
896	8	39.68	298.66667	37.27141	112	51.51800955	-0.02460595	2023	7	(10,[2],[1.0])	0.0
756	9	21.33	252.0	20.09354	84	51.51800955	-0.02460595	2023	7	(10,[0],[1.0])	0.0
600	1	112.2	40.0	105.12132	600	51.51800955	-0.02460595	2023	7	(10,[6],[1.0])	0.0
280	20	8.4	93.33333	10.11712	14	51.51800955	-0.02460595	2023	7	(10,[1],[1.0])	0.0
63	3	1.41	35.7	1.65809	21	51.51800955	-0.02460595	2023	7	(10,[4],[1.0])	0.0

Figure 8

Exploratory Data Analysis

Summary Statistics

Figure 9 below shows summary statistics of all numerical values. The above table shows significant skewness and kurtosis across multiple key variables, including **TOTAL_QUANTITY**, **ITEMS**, **NIC**, **ADQUSAGE**, and **ACTUAL_COST**. These statistical characteristics strongly indicate the presence of extreme outliers in the dataset, which will require careful handling through appropriate transformation techniques such as log scaling to ensure more reliable analytical outcomes.

Further examination reveals potential strong positive correlations between several variables, particularly **TOTAL_QUANTITY**, **NIC**, and **ACTUAL_COST**. This interrelationship between variables necessitates a more detailed investigation through a correlation matrix to fully understand the strength and nature of these relationships and their potential impact on the analysis.

The data also exhibits considerable variability, especially in variables like **NIC** and **ADQUSAGE**. Normalisation and standardisation techniques will be applied to address this high variability and ensure more balanced model inputs. These scaling approaches would help create more uniform variable distributions and potentially improve the overall model performance.

variable	sum	mean	stddev	skewness	kurtosis
TOTAL_QUANTITY	356216.0	1737.639024390244	5148.2668058631125	5.294571133145663	30.07777002957885
ITEMS	4617.0	22.521951219512196	58.27063091769838	4.935970337338197	26.578957215513892
NIC	11017.819999999994	53.745463414634116	140.52586192934444	5.2024070459257405	29.280094199069488
ADQUSAGE	119882.93328000003	584.794796487805	1714.7073838626009	5.29822432802625	30.11504709319609
ACTUAL_COST	10502.586809999999	51.2321307804878	131.89059899762165	5.187997573638253	29.139359092645527
QUANTITY	24697.0	120.47317073170731	117.30196290835069	1.6994744276492795	3.7020764542882194

Figure 9

```

from pyspark.sql.functions import col, mean, stddev, sum, skewness, kurtosis

# List of numerical columns for summary statistics
numerical_columns = ["TOTAL_QUANTITY", "ITEMS", "NIC", "ADQUSAGE", "ACTUAL_COST", "QUANTITY",
                    "PRACTICE_LATITUDE", "PRACTICE_LONGITUDE", "YEAR", "MONTH"]

# Initialize |
statistics = []

# statistics for each column
for column in numerical_columns:
    stats = (
        df_updated
        .select(
            sum(col(column)).alias("sum"),
            mean(col(column)).alias("mean"),
            stddev(col(column)).alias("stddev"),
            skewness(col(column)).alias("skewness"),
            kurtosis(col(column)).alias("kurtosis"),
        )
        .withColumn("variable", lit(column))
    )
    statistics.append(stats)

# statistics into a single DataFrame
summary_stats = statistics[0]
for stat in statistics[1:]:
    summary_stats = summary_stats.union(stat)

# Show summary statistics
summary_stats.select("variable", "sum", "mean", "stddev", "skewness", "kurtosis").show(truncate=False)

```

Screenshot 8

Correlation

The correlation analysis in Figure 10 reveals several significant relationships within the dataset, particularly highlighting that **TOTAL_QUANTITY** and **ADQUSAGE** demonstrate very strong positive correlations with **ACTUAL_COST**, establishing them as powerful predictors. At the same time, **NIC** exhibits an almost perfect correlation (0.9999) with **ACTUAL_COST**, which is logically consistent given the intrinsic relationship between cost and ingredient cost. In contrast, **ITEMS** shows a moderate positive correlation (0.5934), indicating some predictive capability, whereas **QUANTITY** demonstrates only a weak positive correlation (0.1867) with **ACTUAL_COST**, suggesting it may have limited value as a predictive factor in the analysis.

```

Correlations List: [('TOTAL_QUANTITY', 0.991581992425304), ('ITEMS', 0.593422110783414), ('NIC', 0.9999040989393854), ('ADQUSAGE', 0.9895793078247906), ('QUANTITY', 0.186672792929917)]

```

Figure 10

```

from pyspark.sql.functions import col

# Numerical columns to calculate correlation with the target variable
numerical_columns = ["TOTAL_QUANTITY", "ITEMS", "NIC", "ADQUSAGE", "ACTUAL_COST", "QUANTITY",
                    "PRACTICE_LATITUDE", "PRACTICE_LONGITUDE", "YEAR", "MONTH"]
target_variable = "ACTUAL_COST"

# Initialize an empty list to store correlations
correlations = []

# correlation for each column with the target variable
for column in numerical_columns:
    try:
        correlation_value = df_updated.stat.corr(column, target_variable)
        if correlation_value is not None: # Ensure valid correlation value
            correlations.append((column, correlation_value))
        else:
            print(f"Correlation for {column} is None. Skipping.")
    except Exception as e:
        print(f"Error calculating correlation for {column}: {e}")

# Print the correlations List
print("Correlations List:", correlations)

```

Screenshot 9

Feature Engineering

Feature engineering, particularly data scaling and normalisation, is fundamental before preparing the data for machine learning. It ensures optimal algorithm performance, facilitates faster convergence in gradient descent, enables fair comparison of feature importance, and prevents numerical instability (Géron, 2019). Evidence shows that properly engineered features combined with simple models often outperform complex models using raw features (Dong & Liu, 2018). Research further emphasises its importance for distance-based algorithms, neural networks, and support vector machines (Kuhn & Johnson, 2019), though tree-based methods remain invariant to such transformations

Variable	Skewness	Kurtosis	Recommended Transformation	Reason
TOTAL_QUANTITY	6.1724 3282	54.207 85829	Log transformation + Scaling	High skewness and kurtosis, indicating extreme outliers and right-skewed distribution.
ITEMS	6.1544 40065	50.874 98331	Log transformation + Scaling	High skewness and kurtosis suggest outliers and a right-skewed distribution.
NIC	5.3156 11942	43.338 9713	Log transformation + Scaling	High skewness and kurtosis require log transformation to normalize.
ADQUSAGE	6.1360 63109	53.917 69891	Log transformation + Scaling	Right-skewed with high kurtosis. Log transformation reduces skewness.
ACTUAL_COST	5.1882 38851	41.062 65411	Log transformation	Target variable with high skewness and kurtosis. Log transformation normalizes distribution.
QUANTITY	2.3617 9537	10.712 43681	Scaling only	Moderate skewness and kurtosis. Scaling is sufficient without log transformation.
PRACTICE_LATITUDE	- 0.5106 11867	- 0.3468 57309	Cluster K=4	Near-normal distribution. Either retain as is or cluster practices geographically.
PRACTICE_LONGITUDE	- 0.1386 88195	- 1.3194 18662	Cluster K=4	Near-normal distribution. Either retain as is or cluster practices geographically.
YEAR	- 0.1112 67118	- 1.9876 19628	Scaling only	Scale for machine learning model

	-	-		
	0.0559	1.0850		
MONTH	65039	33158	Scaling Only	Scale for machine learning model

Log Transformation for ACTUAL_COST

Log Transformation compresses large values and spreads smaller ones to normalise the distribution as the values Range very high between the Min and Max Ranges. *Figure 12* below shows the transformed Logged Data. The original variable column will not be dropped as the variable will be used to compare the model with its Log value.

```
+-----+-----+
|ACTUAL_COST|LOG_ACTUAL_COST|
+-----+-----+
|415.30817|6.031425779278594|
|3.70971|1.5496263350074813|
|136.4584|4.923321325875971|
|37.02111|3.6381415317945547|
|1.56749|0.9429287679736535|
|1.69559|0.9916171043338229|
|6.67305|2.0377141895303503|
|7.98134|2.195149091671816|
|3.12257|1.41647675529533|
|15.96268|2.831015636748015|
|10.19225|2.41522157449755|
|4.15933|1.6408067261016441|
|11.46614|2.5230161688706847|
|40.07547|3.7154111063411293|
|3.70954|1.54959023871531|
|11.2415|2.50483181859527|
|22.33937|3.150141634206194|
|57.88679|4.07561678706534|
|24.931|3.2554391641752476|
|6.98219|2.0772128099012477|
+-----+-----+
only showing top 20 rows
```

Figure 12

```
from pyspark.sql.functions import log, col

# Apply log transformation to the target variable
df_encoded = df_encoded.withColumn("LOG_ACTUAL_COST", log(col("ACTUAL_COST") + 1))

# Display transformed target variable
df_encoded.select("ACTUAL_COST", "LOG_ACTUAL_COST").show(truncate=False)
```

Screenshot 10

Log Transformation + Scaling for Input Variables

Handles skewness (TOTAL_QUANTITY, NIC, ADQUSAGE) and ensures all features contribute equally to models that rely on magnitude. *Figure 13* shows the transformation

LOG_TOTAL_QUANTITY	TOTAL_QUANTITY_scaled	LOG_NIC	NIC_scaled	LOG_ADQUSAGE	ADQUSAGE_scaled
9.660205067381032	[2.541626137164313]	6.094472297182211	[2.455755774083107]	8.56172031219105	[2.461121069971588]
4.727387818712341	[-0.8226463058090829]	1.5769147207285403	[-0.9202412683543075]	3.6463197527386146	[-0.9033930524759506]
8.547334348328224	[1.7826277587606107]	4.986069340151007	[1.627439799226375]	7.449110105843086	[1.6995569396916341]
6.970730078143525	[0.707354536123367]	3.4468078929142076	[0.47714077097507374]	5.873993972005715	[0.6214147999631158]
4.04305126783455	[-1.2893764690761236]	0.9783261227936078	[-1.3675700078479758]	2.9789253247291207	[-1.3602140075785147]
4.110873864173311	[-1.2431202060988191]	0.9895411936137477	[-1.359188920218487]	3.044522437723423	[-1.3153138187959952]
5.53338948872752	[-0.2729382848694391]	2.0930978681273213	[-0.5344946011602382]	4.442651256490317	[-0.35831670353625167]
5.53338948872752	[-0.2729382848694391]	2.089391872533004	[-0.5372641132149186]	4.442651256490317	[-0.35831670353625167]
4.727387818712341	[-0.8226463058090829]	1.463255402256019	[-1.005179537876022]	3.6463197527386146	[-0.9033930524759506]
6.22455842927536	[0.19845169430560142]	2.7186603802142257	[-0.0670080987916354]	5.1298987149230735	[0.1120933762848467]
5.41610040220442	[-0.3529316075001699]	2.3905959703167587	[-0.3121725397089016]	4.326337772866156	[-0.4379314477525057]
4.948759890378168	[-0.6716664655260729]	1.601405740736836	[-0.9019389867087491]	3.864232411521865	[-0.7542352760000471]
5.53338948872752	[-0.2729382848694391]	2.4051416813191375	[-0.3013024453065345]	5.1298987149230735	[0.1120933762848467]
7.321849713788356	[0.9468246106949664]	3.7764323511217976	[0.7234710450647793]	6.22455842927536	[0.8613706393796435]
4.442651256490317	[-1.0168418986113643]	1.2584609896100059	[-1.158223593142583]	3.367295829986474	[-1.0943805248590488]
6.22455842927536	[0.19845169430560142]	2.5079719227189963	[-0.22445680804261092]	5.6580875153940715	[0.47363023721765324]
5.820082930352362	[-0.07740806418267711]	3.211649779776461	[0.30140574579330076]	5.41610040220442	[0.3079938765733053]
7.68937110752969	[1.197480987129923]	4.1370847158982595	[0.9929886541329062]	6.591673732008658	[1.1126552713589302]
6.799055862058796	[0.5902695509662966]	3.3163654461454577	[0.37966037209871956]	5.702670756926981	[0.5041467625644042]
5.1298987149230735	[-0.5481264426449319]	2.132982308607866	[-0.5046887268968068]	4.04305126783455	[-0.6318365904510166]

Figure 13

```

from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.sql.functions import log, col

# Step 1: Drop existing conflicting columns
columns_to_drop = [
    "TOTAL_QUANTITY_vec", "TOTAL_QUANTITY_scaled",
    "NIC_vec", "NIC_scaled",
    "ADQUSAGE_vec", "ADQUSAGE_scaled"
]
df_encoded = df_encoded.drop(*columns_to_drop)

# Step 2: Log-transform the selected columns
log_columns = ["TOTAL_QUANTITY", "NIC", "ADQUSAGE"]
for col_name in log_columns:
    df_encoded = df_encoded.withColumn(f"LOG_{col_name}", log(col(col_name) + 1))

# Step 3: Apply separate scaling for each log-transformed column
for col_name in log_columns:
    # Assemble the column into a vector
    assembler = VectorAssembler(inputCols=[f"LOG_{col_name}"], outputCol=f"{col_name}_vec")
    df_encoded = assembler.transform(df_encoded)

    # Apply standard scaling
    scaler = StandardScaler(inputCol=f"{col_name}_vec", outputCol=f"{col_name}_scaled", withMean=True, withStd=True)
    scaler_model = scaler.fit(df_encoded)
    df_encoded = scaler_model.transform(df_encoded)

# Step 4: Display scaled columns
df_encoded.select(
    "LOG_TOTAL_QUANTITY", "TOTAL_QUANTITY_scaled",
    "LOG_NIC", "NIC_scaled",
    "LOG_ADQUSAGE", "ADQUSAGE_scaled"
).show(truncate=False)

```

Screenshot 11

Scaling QUANTITY, YEAR, and MONTH

	QUANTITY	QUANTITY_scaled	YEAR	YEAR_scaled	MONTH	MONTH_scaled
224		[0.1859883236030025]	2023	[0.0]	7	[0.5454545454545454]
112		[0.09257714762301918]	2023	[0.0]	7	[0.5454545454545454]
224		[0.1859883236030025]	2023	[0.0]	7	[0.5454545454545454]
14		[0.010842368640533777]	2023	[0.0]	7	[0.5454545454545454]
56		[0.04587155963302752]	2023	[0.0]	7	[0.5454545454545454]
60		[0.04920767306088407]	2023	[0.0]	7	[0.5454545454545454]
252		[0.2093411175979983]	2023	[0.0]	7	[0.5454545454545454]
21		[0.016680567139282735]	2023	[0.0]	7	[0.5454545454545454]
112		[0.09257714762301918]	2023	[0.0]	7	[0.5454545454545454]
21		[0.016680567139282735]	2023	[0.0]	7	[0.5454545454545454]
28		[0.022518765638031693]	2023	[0.0]	7	[0.5454545454545454]
35		[0.028356964136780648]	2023	[0.0]	7	[0.5454545454545454]
14		[0.010842368640533777]	2023	[0.0]	7	[0.5454545454545454]
168		[0.13928273561301083]	2023	[0.0]	7	[0.5454545454545454]
7		[0.0050041701417848205]	2023	[0.0]	7	[0.5454545454545454]
84		[0.06922435362802334]	2023	[0.0]	7	[0.5454545454545454]
168		[0.13928273561301083]	2023	[0.0]	7	[0.5454545454545454]
168		[0.13928273561301083]	2023	[0.0]	7	[0.5454545454545454]
224		[0.1859883236030025]	2023	[0.0]	7	[0.5454545454545454]
168		[0.13928273561301083]	2023	[0.0]	7	[0.5454545454545454]

Figure13

```

from pyspark.ml.feature import MinMaxScaler

# List of individual columns to scale
columns_to_scale = ["QUANTITY", "YEAR", "MONTH"]

# Scale each column separately
for column in columns_to_scale:
    # Assemble the column into a vector
    assembler = VectorAssembler(inputCols=[column], outputCol=f"{column}_vec")
    df_encoded = assembler.transform(df_encoded)

    # Apply Min-Max Scaling
    scaler = MinMaxScaler(inputCol=f"{column}_vec", outputCol=f"{column}_scaled")
    scaler_model = scaler.fit(df_encoded)
    df_encoded = scaler_model.transform(df_encoded)

# Display scaled columns
df_encoded.select("QUANTITY", "QUANTITY_scaled", "YEAR", "YEAR_scaled", "MONTH", "MONTH_scaled").show(truncate=False)

```

Screenshot 12

Clustering Geo-Positions

As the dataset contains information on the London Borough of Tower Hamlets, the geo-positions of Latitude and Longitude can be clustered by K=4 as the borough contains four types of Postcodes starting E1, E2, E3 and E14. This will later help to analyse which area spends more on Metformin Prescriptions. Figure 15 below shows the clustered data

PRACTICE_LATITUDE	PRACTICE_LONGITUDE	geo_cluster
51.5135999	-0.05055	1
51.51835505	-0.03830635	2
51.5363018	-0.027021075	2
51.5238708	-0.014907	3
51.52979	-0.03887	2
51.5141762	-0.0143071	0
51.5141762	-0.0143071	0
51.50533401	-0.058222367	1
51.5139726	-0.0550705	1
51.52468015	-0.04266905	2
51.5173976	-0.0696235	1
51.52377	-0.0651999	1
51.5269199	-0.06426	1
51.5269199	-0.06426	1
51.5238708	-0.014907	3
51.52589465	-0.02501745	3
51.51448982	-0.004900067	0
51.50071373	-0.025713573	0
51.5135999	-0.05055	1
51.5139726	-0.0550705	1

Figure 15

```
from pyspark.ml.clustering import KMeans

# Assemble Latitude and Longitude for clustering
geo_assembler = VectorAssembler(inputCols=["PRACTICE_LATITUDE", "PRACTICE_LONGITUDE"], outputCol="geo_features")
df_encoded = geo_assembler.transform(df_encoded)

# Apply KMeans clustering with k=4
kmeans = KMeans(featuresCol="geo_features", predictionCol="geo_cluster", k=4, seed=42)
kmeans_model = kmeans.fit(df_encoded)
df_encoded = kmeans_model.transform(df_encoded)

# Display geolocation clusters
df_encoded.select("PRACTICE_LATITUDE", "PRACTICE_LONGITUDE", "geo_cluster").show()
```

Screenshot 13

Machine Learning Phase

Pipeline

Transformation, encoding, and all other feature engineering processes shown previously will be implemented through a pipeline including train-test split, model training and evaluation. The reason for following this method is that a pipeline provides a structured and reusable framework for sequentially applying transformations and modelling steps. This approach offers significant advantages, particularly in automated preprocessing and model training scenarios for multiple datasets or production workflows. The analysis involves two distinct pipelines to compare modelling approaches, one utilising **LOG_ACTUAL_COST** as the target variable to address data skewness and outliers, while the other uses the original **ACTUAL_COST** for comparison. Both pipelines implement an 80-20 train-test split of the dataset. *Screenshot 13* below partially shows how the pipelines have been coded. For full code, refer to the ipynb file.

```

log_target_column = "LOG_ACTUAL_COST"
df_updated = df_updated.withColumn(log_target_column, log(col("ACTUAL_COST") + 1))

#StringIndexer and OneHotEncoder for categorical variables
indexer = StringIndexer(inputCol="BNF_DESCRIPTION", outputCol="BNF_DESCRIPTION_INDEX")
encoder = OneHotEncoder(inputCol="BNF_DESCRIPTION_INDEX", outputCol="BNF_DESCRIPTION_OHE")
chem_indexer = StringIndexer(inputCol="CHEMICAL_SUBSTANCE_BNF_DESCR", outputCol="CHEMICAL_SUBSTANCE_BNF_DESCR_INDEX")

#Log-transform numerical features
log_columns = ["TOTAL_QUANTITY", "NIC", "ADQUUSAGE"]
for col_name in log_columns:
    df_updated = df_updated.withColumn(f"LOG_{col_name}", log(col(col_name) + 1))

#Assemble and scale log-transformed features
scaled_features = []
for col_name in log_columns:
    assembler = VectorAssembler(inputCols=[f"LOG_{col_name}"], outputCol=f"{col_name}_vec")
    scaler = StandardScaler(inputCol=f"{col_name}_vec", outputCol=f"{col_name}_scaled", withMean=True, withStd=True)
    scaled_features.append(assembler)
    scaled_features.append(scaler)

#MinMax scaling for QUANTITY, YEAR, MONTH
minmax_columns = ["QUANTITY", "YEAR", "MONTH"]
minmax_scalers = []
for column in minmax_columns:
    assembler = VectorAssembler(inputCols=[column], outputCol=f"{column}_vec")
    scaler = MinMaxScaler(inputCol=f"{column}_vec", outputCol=f"{column}_scaled")
    minmax_scalers.append(assembler)
    minmax_scalers.append(scaler)

#KMeans clustering for geolocation

```

Screenshot 14

Model Training

Linear Regression is an appropriate and effective model for this dataset, given the nature of the target variable **ACTUAL_COST** or its log-transformed version, which is continuous and aligns well with the algorithm's purpose of predicting numerical outcomes. The simplicity and interpretability of Linear Regression make it particularly valuable, as the coefficients provide clear insights into the relationships between features and the target variable, which is useful in business and operational contexts. *Screenshot 14 below* shows the code used to train the model with Linear Regression

```

#Linear Regression
lr = LinearRegression(featuresCol="features", labelCol=log_target_column)

# Pipeline assembly
stages = [
    indexer, encoder, chem_indexer,
    *scaled_features, *minmax_scalers,
    geo_assembler, feature_assembler, lr
]
log_pipeline = Pipeline(stages=stages)

# Train-test split
train_data, test_data = df_updated.randomSplit([0.8, 0.2], seed=42)

# Fit the pipeline and evaluate the model
log_model = log_pipeline.fit(train_data)
log_predictions = log_model.transform(test_data)

```

Screenshot 15

Model Evaluation

The model's performance is assessed through multiple evaluation metrics: Root Mean Squared Error (RMSE) measures the accuracy of predictions by quantifying the typical deviation between predicted and actual values, Mean Squared Error (MSE) calculates the average squared difference between predictions and actual values, and R^2 (R-Squared) indicates how much of the variance in the target variable is explained by the model's predictions. Figures 18 and 19 show the model results from the Log Pipeline and Original Target pipeline.

```
Original Target Variable Metrics:  
Root Mean Squared Error (RMSE): 56.24526007473526  
 $R^2$ : 0.6300262836881126
```

Figure 18

```
Root Mean Squared Error (RMSE): 0.09353342859553959  
R-squared ( $R^2$ ): 0.9948099942291405
```

Figure 4

Analysis of results

1. Log-Transformed Target Variable:

RMSE: 0.09350.09350.0935

R^2 : 0.9950.9950.995

This indicates that the model explains 99.5% of the variance in the log-transformed target variable. The low RMSE signifies that the model's predictions are very close to the actual log-transformed values, suggesting strong performance.

2. Original Target Variable:

RMSE: 56.2556.2556.25

R^2 : 0.630.630.63

The lower R^2 suggests that the model explains only 63% of the variance in the original target variable. The higher RMSE reflects larger prediction errors when directly modelling the original target variable.

The log transformation significantly enhances model performance by stabilising variance and mitigating the influence of extreme values (outliers) in the target variable, **ACTUAL_COST**. This improvement is evidenced by the high R^2 and low RMSE observed for the log-transformed target variable, underscoring the effectiveness of this technique. In contrast, the model's poorer performance with the original target variable highlights the challenges posed by skewness and the disproportionate impact of outliers. These issues were addressed through log transformation, validating its suitability when the target variable exhibits a wide range of heavy skewness. This demonstrates the critical role of transformation in improving model accuracy and reliability for such datasets.

Data Visualisation by Power BI

Total number of Prescriptions by GP Practice

Figure 20 Below shows the Highest number of prescriptions was prescribed by Goodman's Field Health Centre; the amount was 1.72 million prescriptions

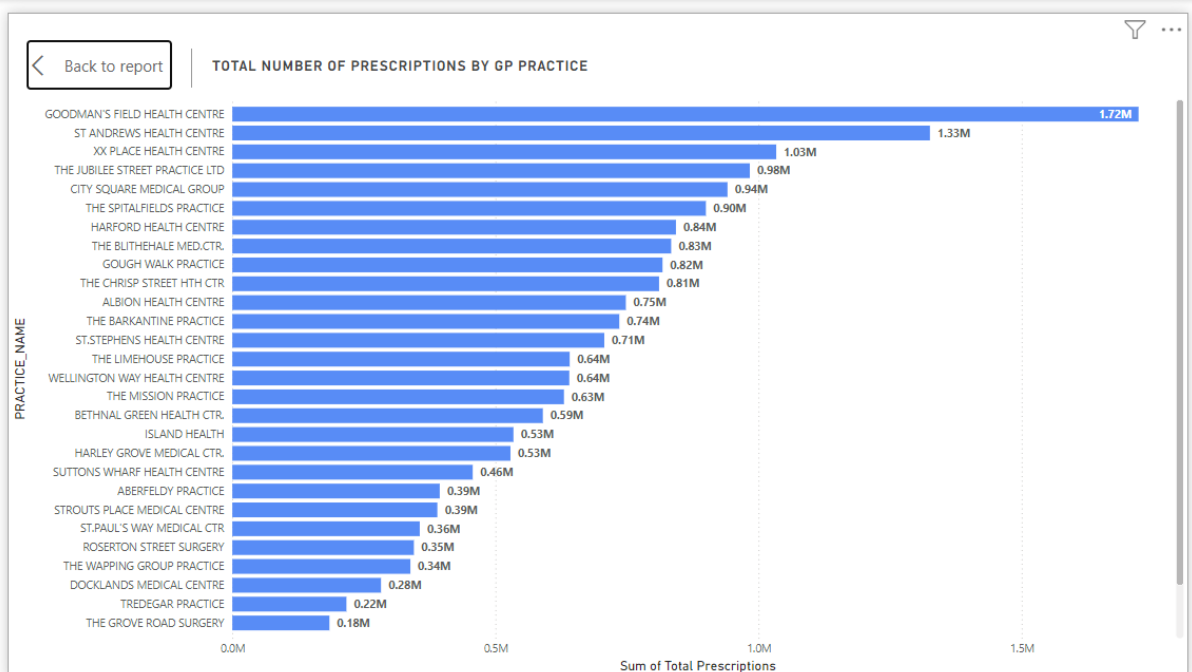


Figure 20

Time series chart for Actual Cost

Figure 21 shows the Cost of prescription was highest during the month of August (51.5K) and the lowest in June (40.5K)



Figure 21

Geospatial data of the total number of prescriptions

When Clicking on a blue bubble in Figure 22, the Practice name and Total number of Prescription for Metformin will appear

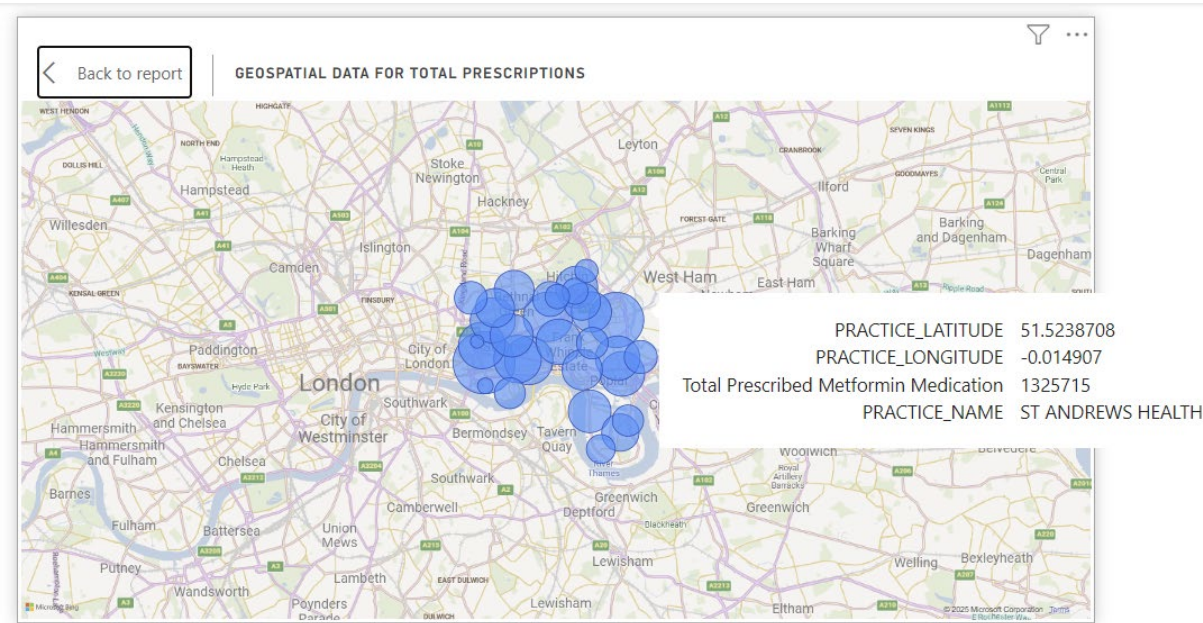


Figure 22

Monthly Total Cost of Prescription Vs Actual Cost

Figure 23 shows the total number of Prescriptions vs Actual Cost as an additional feature. The highest number of prescriptions was issued in July with the highest cost to the NHS, however, for the months of June and August, the total actual cost differed despite a similar number of Prescriptions issued for both months.

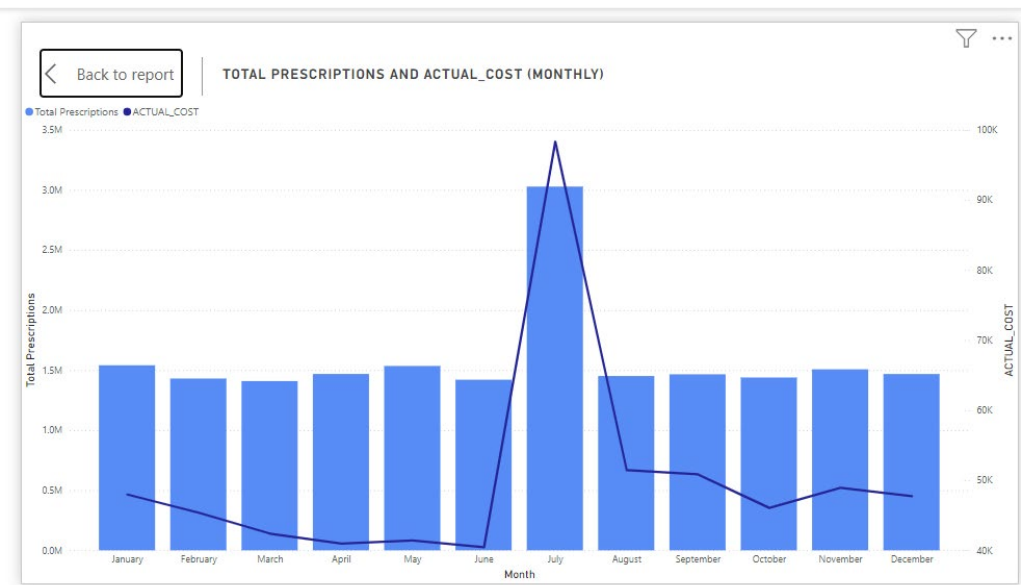


Figure 23

Interactive Dashboard

Figure 24 shows an Interactive dashboard with a slicer at the bottom right corner. When a practice is chosen from the drop-down menu, individual figures change for all the visualisations indicating the particular practice's stats.

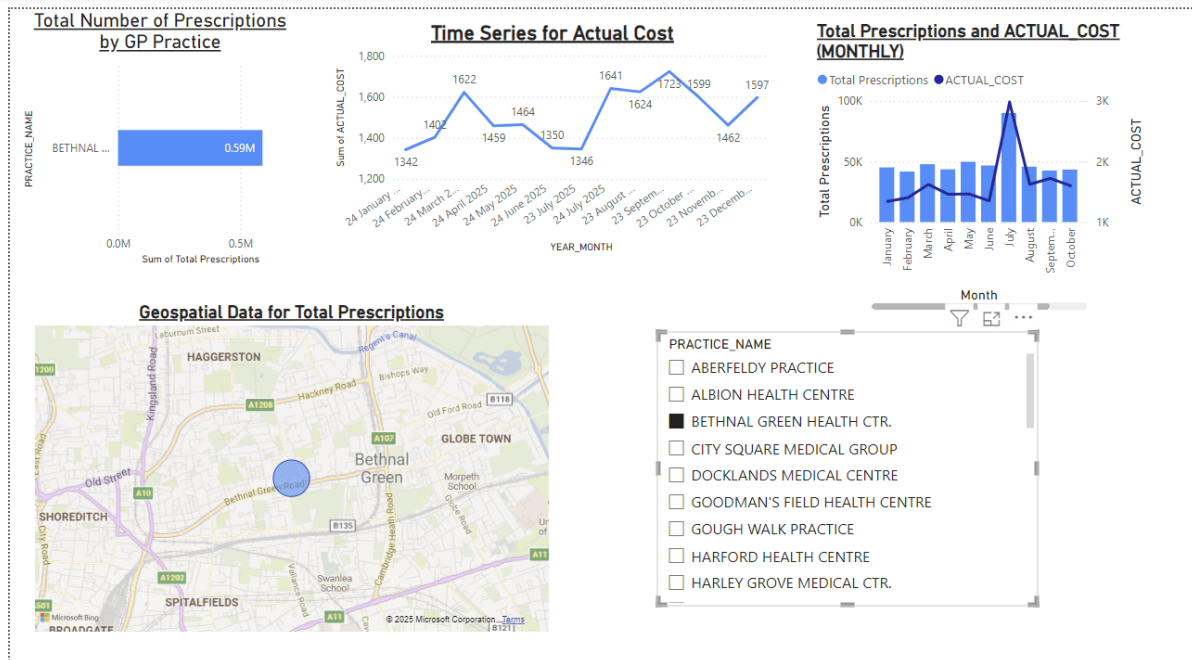
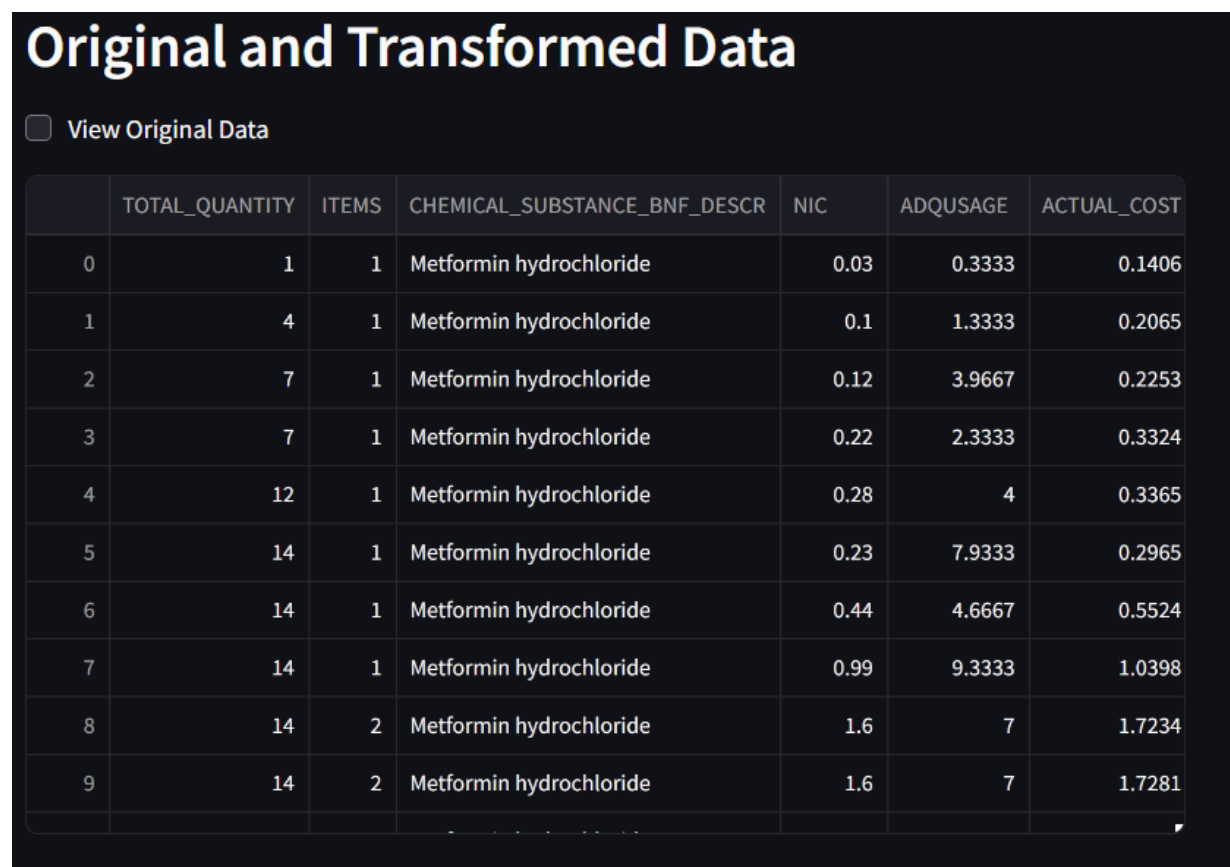


Figure 24

StreamLit Interactive Dashboard

Feature 1

A key dashboard feature is its ability to toggle between the **transformed dataset** (used for machine learning modelling) and the **original dataset** (reversed from transformations such as scaling and log transformations). This feature uses a checkbox, allowing users to view and compare both datasets in a tabular format. This comparison is crucial for debugging and understanding the effects of transformations on the dataset. For the whole code to setup streamlit dashboard please refer to Appendix 3



	TOTAL_QUANTITY	ITEMS	CHEMICAL_SUBSTANCE_BNF_DESCR	NIC	ADQUSAGE	ACTUAL_COST
0	1	1	Metformin hydrochloride	0.03	0.3333	0.1406
1	4	1	Metformin hydrochloride	0.1	1.3333	0.2065
2	7	1	Metformin hydrochloride	0.12	3.9667	0.2253
3	7	1	Metformin hydrochloride	0.22	2.3333	0.3324
4	12	1	Metformin hydrochloride	0.28	4	0.3365
5	14	1	Metformin hydrochloride	0.23	7.9333	0.2965
6	14	1	Metformin hydrochloride	0.44	4.6667	0.5524
7	14	1	Metformin hydrochloride	0.99	9.3333	1.0398
8	14	2	Metformin hydrochloride	1.6	7	1.7234
9	14	2	Metformin hydrochloride	1.6	7	1.7281

Figure 24.1

Feature 2

The dashboard also includes options for **data visualization** to assist with exploratory data analysis (EDA). Users can select one or more columns to create either a **line chart** (for multiple columns) or a **bar chart** (for a single column). These visualizations offer insights into the dataset's trends, distributions, and relationships, providing an intuitive way to interpret the data.

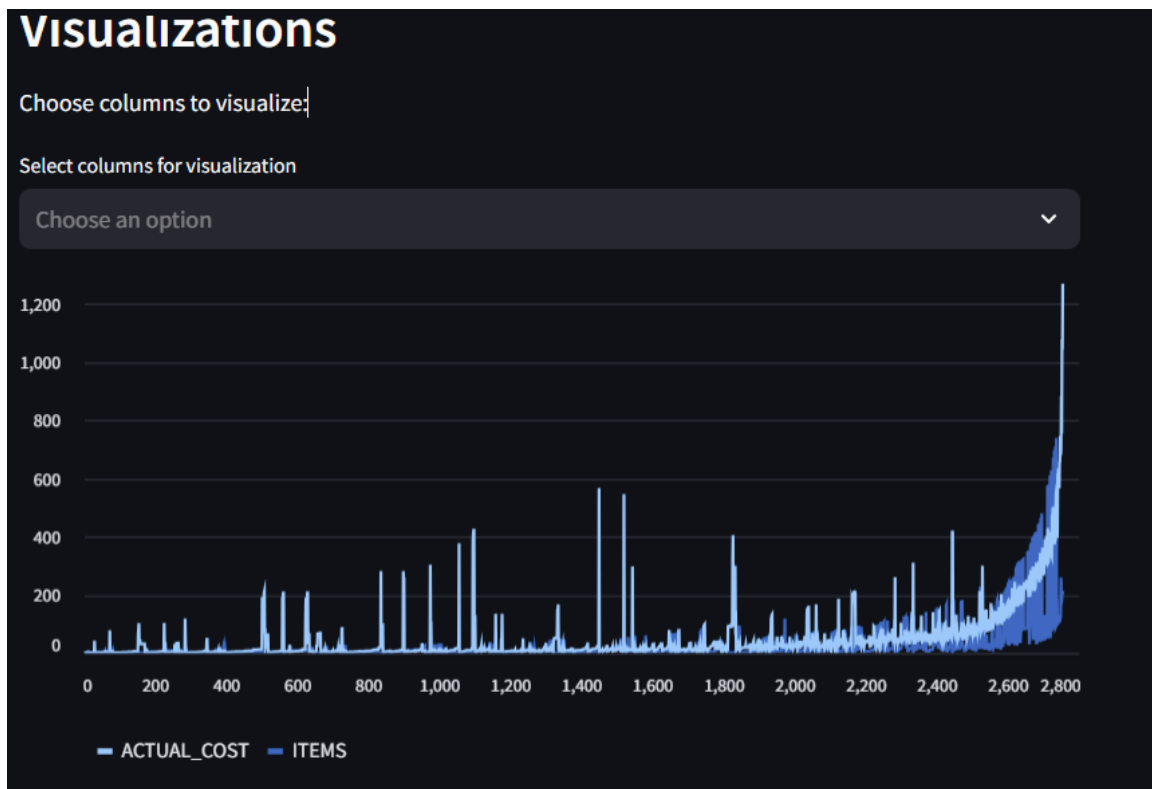


Figure 24.2

Feature 3

Additionally, the **search and filter functionality** enables users to search for specific rows of data by selecting a column and entering a keyword or numeric value. The results dynamically display rows matching the input, allowing users to locate and focus on data points of interest.

Search and Filter

Select a column to search

ITEMS



Enter a value to filter

23

	TOTAL_QUANTITY	ITEMS	CHEMICAL_SUBSTANCE_BNF_DESCR	NIC	ADQUSAGE	ACTUAL_COST
733	161	23	Metformin hydrochloride	3.91	53.6667	5.7143
734	161	23	Metformin hydrochloride	3.91	53.6667	6.271
735	161	23	Metformin hydrochloride	4.14	53.6667	6.4632
736	161	23	Metformin hydrochloride	4.14	53.6667	6.4778
1,346	322	23	Metformin hydrochloride	5.29	182.4667	6.819
1,347	322	23	Metformin hydrochloride	8.28	107.3333	10.3648
1,348	322	23	Metformin hydrochloride	10.12	107.3333	12.7052
1,349	322	23	Metformin hydrochloride	22.77	214.6667	23.9116
2,359	1,932	23	Metformin hydrochloride	45.54	644	36.7305
2,360	1,932	23	Metformin hydrochloride	45.54	644	36.7327

Figure 24.3

PART 2

Real-Time Stock Data Streaming Using Kafka

Overview

This part of the report will demonstrate the use of Kafka for real-time stock data streaming using the Alpha Vantage API. The key objectives include:

- Writing a Kafka producer to stream live-stock data.
- Creating a Kafka consumer to display and process the data.
- Extending the consumer to calculate and display descriptive statistics.
- Saving streamed data into a CSV file for permanent storage.

Writing Kafka Producer

The producer script (`producer.py`) was implemented to fetch live-stock data from the Alpha Vantage API and send it to the Kafka topic stock stream. The script used the `requests` library to connect to the Alpha Vantage API, which provides intraday stock data for IBM. It fetched stock prices in 5-minute intervals and formatted the data into JSON messages containing key details such as the symbol, timestamp, open price, high price, low price, close price, and trading volume. These messages were published to the Kafka topic using the `Kafka-python` library. The producer script ensured that live-stock data was continuously streamed by fetching new data and sending it to Kafka every 5 minutes. This setup successfully integrated real-time data fetching and publishing. The script was executed using the command `python producer.py`, and its execution was verified by observing logs in the terminal *Figure25* showing messages being sent to the Kafka topic.

```

import requests
import json
from kafka import KafkaProducer
import time

# Kafka Producer Configuration
producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

# Kafka Topic
topic = "stock-stream"

# Alpha Vantage API Configuration
ALPHA_VANTAGE_API_KEY = "TGVODHVXA5GLTY5X" # Replace with API key
symbol = "IBM"
url = f'https://www.alphavantage.co/query?function=TIME_SERIES_INTRADAY&symbol={symbol}&interval=5min&apikey={ALPHA_VANTAGE_API_KEY}'

while True:
    try:
        response = requests.get(url)
        data = response.json()
        time_series = data.get("Time Series (5min)", {})

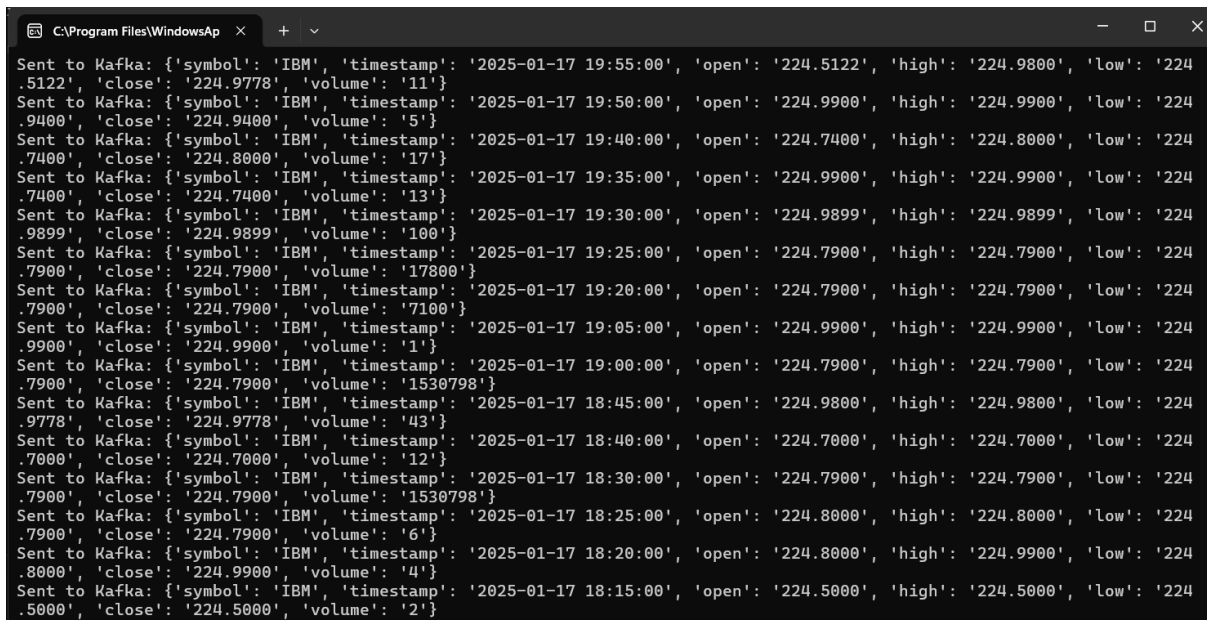
        # Send each data point to Kafka
        for timestamp, stock_data in time_series.items():
            message = {
                "symbol": symbol,
                "timestamp": timestamp,
                "open": stock_data["1. open"],
                "high": stock_data["2. high"],
                "low": stock_data["3. low"],
                "close": stock_data["4. close"],
                "volume": stock_data["5. volume"]
            }
            producer.send(topic, message)
            print(f"Sent to Kafka: {message}")

            time.sleep(300) # Alpha Vantage updates every 5 minutes
    except Exception as e:
        print(f"Error: {e}")
        break

producer.close()

```

Figure 25



```

C:\Program Files\WindowsAp
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:55:00', 'open': '224.5122', 'high': '224.9800', 'low': '224.5122', 'close': '224.9778', 'volume': '11'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:50:00', 'open': '224.9900', 'high': '224.9900', 'low': '224.9400', 'close': '224.9400', 'volume': '5'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:40:00', 'open': '224.7400', 'high': '224.8000', 'low': '224.7400', 'close': '224.8000', 'volume': '17'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:35:00', 'open': '224.9900', 'high': '224.9900', 'low': '224.7400', 'close': '224.7400', 'volume': '13'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:30:00', 'open': '224.9899', 'high': '224.9899', 'low': '224.9899', 'close': '224.9899', 'volume': '100'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:25:00', 'open': '224.7900', 'high': '224.7900', 'low': '224.7900', 'close': '224.7900', 'volume': '17800'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:20:00', 'open': '224.7900', 'high': '224.7900', 'low': '224.7900', 'close': '224.7900', 'volume': '7100'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:05:00', 'open': '224.9900', 'high': '224.9900', 'low': '224.9900', 'close': '224.9900', 'volume': '1'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 19:00:00', 'open': '224.7900', 'high': '224.7900', 'low': '224.7900', 'close': '224.7900', 'volume': '1530798'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 18:45:00', 'open': '224.9800', 'high': '224.9800', 'low': '224.9778', 'close': '224.9778', 'volume': '43'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 18:40:00', 'open': '224.7000', 'high': '224.7000', 'low': '224.7000', 'close': '224.7000', 'volume': '12'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 18:30:00', 'open': '224.7900', 'high': '224.7900', 'low': '224.7900', 'close': '224.7900', 'volume': '1530798'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 18:25:00', 'open': '224.8000', 'high': '224.8000', 'low': '224.7900', 'close': '224.7900', 'volume': '6'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 18:20:00', 'open': '224.8000', 'high': '224.9900', 'low': '224.8000', 'close': '224.9900', 'volume': '4'}
Sent to Kafka: {'symbol': 'IBM', 'timestamp': '2025-01-17 18:15:00', 'open': '224.5000', 'high': '224.5000', 'low': '224.5000', 'close': '224.5000', 'volume': '2'}

```

Figure 26

Writing the Kafka Consumer

The consumer script (consumer.py) was designed to read and display the stock data messages sent by the producer to the Kafka topic stock-stream. Using the kafka-python library, the consumer was configured to connect to the Kafka broker and subscribe to the topic. The consumer displayed details such as the timestamp, open price, high price, low price, close price, and trading volume of IBM stock. This script (*Figure 27*) confirmed that data was successfully streamed from the producer to the Kafka topic and then consumed in real time. The consumer was executed using the command `python consumer.py`, and its functionality was validated by the continuous display of stock data in the terminal.

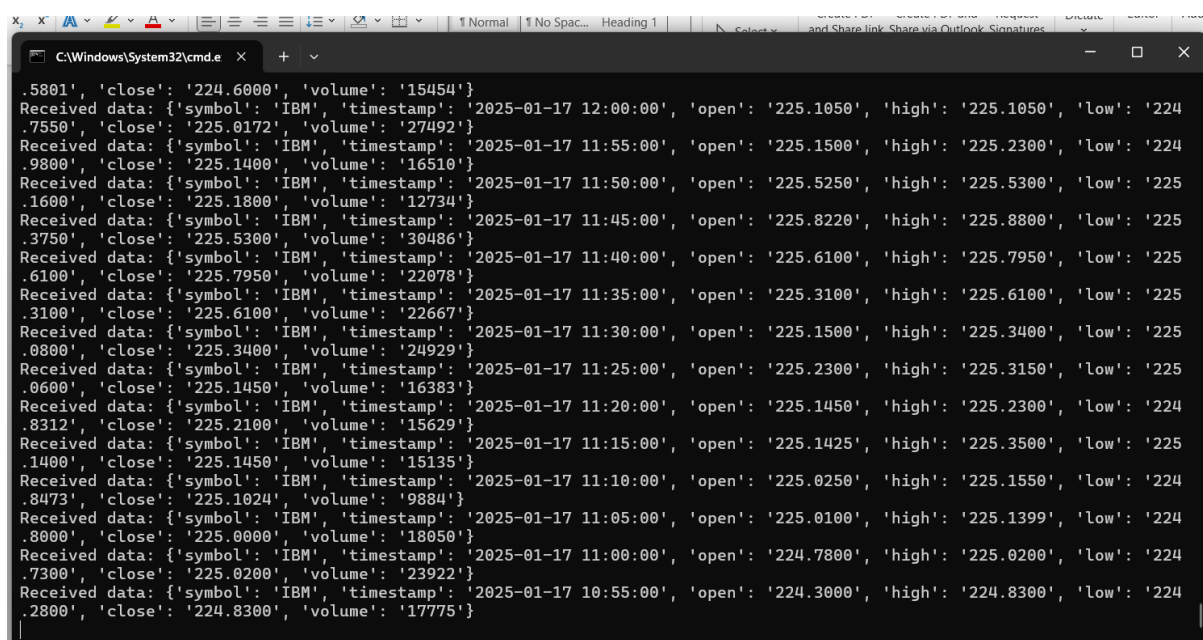
```
from kafka import KafkaConsumer
import json

# Kafka Consumer Configuration
topic = "stock-stream" # Replace with your Kafka topic
consumer = KafkaConsumer(
    topic,
    bootstrap_servers='localhost:9092', # Kafka broker address
    auto_offset_reset='earliest',       # Start reading messages from the beginning
    value_deserializer=lambda v: json.loads(v.decode('utf-8')) # Deserialize JSON messages
)

print(f"Listening for messages on Kafka topic '{topic}'...")

# Consume messages
for message in consumer:
    stock_data = message.value # Extract the message value
    print(f"Received data: {stock_data}")
```

Figure 27



```
C:\Windows\System32\cmd.exe
.5801', 'close': '224.6000', 'volume': '15454'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 12:00:00', 'open': '225.1050', 'high': '225.1050', 'low': '224
.7550', 'close': '225.0172', 'volume': '27492'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:55:00', 'open': '225.1500', 'high': '225.2300', 'low': '224
.9800', 'close': '225.1400', 'volume': '16510'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:50:00', 'open': '225.5250', 'high': '225.5300', 'low': '225
.1600', 'close': '225.1800', 'volume': '12734'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:45:00', 'open': '225.8220', 'high': '225.8800', 'low': '225
.3750', 'close': '225.5300', 'volume': '30486'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:40:00', 'open': '225.6100', 'high': '225.7950', 'low': '225
.6100', 'close': '225.7950', 'volume': '22078'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:35:00', 'open': '225.3100', 'high': '225.6100', 'low': '225
.3100', 'close': '225.6100', 'volume': '22667'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:30:00', 'open': '225.1500', 'high': '225.3400', 'low': '225
.0800', 'close': '225.3400', 'volume': '24929'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:25:00', 'open': '225.2300', 'high': '225.3150', 'low': '225
.0600', 'close': '225.1450', 'volume': '16383'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:20:00', 'open': '225.1450', 'high': '225.2300', 'low': '224
.8312', 'close': '225.2100', 'volume': '15629'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:15:00', 'open': '225.1425', 'high': '225.3500', 'low': '225
.1400', 'close': '225.1450', 'volume': '15135'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:10:00', 'open': '225.0250', 'high': '225.1550', 'low': '224
.8473', 'close': '225.1024', 'volume': '9884'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:05:00', 'open': '225.0100', 'high': '225.1399', 'low': '224
.8000', 'close': '225.0000', 'volume': '18050'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:00:00', 'open': '224.7800', 'high': '225.0200', 'low': '224
.7300', 'close': '225.0200', 'volume': '23922'}
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 10:55:00', 'open': '224.3000', 'high': '224.8300', 'low': '224
.2800', 'close': '224.8300', 'volume': '17775'}
```

Figure 28

Extending the Consumer to Display Descriptive Statistics

As shown in *Figure 29* and *Figure 30*, the consumer script was extended to calculate and display descriptive statistics for the streamed data to enhance functionality. Key statistics included the **average volume**, **maximum close price**, and **minimum open price**. The script processed messages in batches of 10, accumulating the relevant fields from each message, and computed the statistics once the batch was complete. For instance, after every 10 messages, the average trading volume was calculated using Python's statistics library. At the same time, the maximum and minimum values for close and open prices, respectively, were determined using built-in functions.

```
from kafka import KafkaConsumer
import json
import statistics

# Kafka Consumer Configuration
topic = "stock-stream" # Kafka topic name
consumer = KafkaConsumer(
    topic,
    bootstrap_servers='localhost:9092', # Kafka broker address
    auto_offset_reset='earliest', # Start reading from the beginning
    value_deserializer=lambda v: json.loads(v.decode('utf-8')) # Deserialize JSON messages
)

print(f"Listening for messages on Kafka topic '{topic}'...")

# Lists to store numeric data for statistics
volumes = []
close_prices = []
open_prices = []

for message in consumer:
    stock_data = message.value # Extract the message value
    print(f"Received data: {stock_data}")

    # Extract numeric fields for statistics
    volume = float(stock_data["volume"])
    close_price = float(stock_data["close"])
    open_price = float(stock_data["open"])

    # Append to lists
    volumes.append(volume)
    close_prices.append(close_price)
    open_prices.append(open_price)

    # Display descriptive statistics every 1 messages
    if len(volumes) >= 1:
        avg_volume = statistics.mean(volumes)
        max_close = max(close_prices)
        min_open = min(open_prices)

        print("\n=== Descriptive Statistics ===")
        print(f"Average Volume: {avg_volume}")
        print(f"Maximum Close Price: {max_close}")
        print(f"Minimum Open Price: {min_open}")
        print("=====\n")

    # Clear lists after displaying statistics
    volumes.clear()
    close_prices.clear()
    open_prices.clear()
```

Figure 29

```
C:\Program Files\WindowsAp x + v
=====
Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:05:00', 'open': '225.0100', 'high': '225.1399', 'low': '224.8000', 'close': '225.0000', 'volume': '18050'}

=== Descriptive Statistics ===
Average Volume: 18050.0
Maximum Close Price: 225.0
Minimum Open Price: 225.01
=====

Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 11:00:00', 'open': '224.7800', 'high': '225.0200', 'low': '224.7300', 'close': '225.0200', 'volume': '23922'}

=== Descriptive Statistics ===
Average Volume: 23922.0
Maximum Close Price: 225.02
Minimum Open Price: 224.78
=====

Received data: {'symbol': 'IBM', 'timestamp': '2025-01-17 10:55:00', 'open': '224.3000', 'high': '224.8300', 'low': '224.2800', 'close': '224.8300', 'volume': '17775'}

=== Descriptive Statistics ===
Average Volume: 17775.0
Maximum Close Price: 224.83
Minimum Open Price: 224.3
=====
```

Figure 30

Saving file to .csv file

Figure 31 below shows the code for capturing the stock price and saving it to a .csv file

```
from kafka import KafkaConsumer
import json
import csv

# Kafka Consumer Configuration
topic = "stock-stream" # Kafka topic name
consumer = KafkaConsumer(
    topic,
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
)

print(f"Listening for messages on Kafka topic '{topic}'...")

# Open a CSV file in append mode
with open('stock_data.csv', 'a', newline='') as file:
    writer = csv.writer(file)

    # Write header if the file is empty
    if file.tell() == 0:
        writer.writerow(["symbol", "timestamp", "open", "high", "low", "close", "volume"])

    for message in consumer:
        stock_data = message.value
        print(f"Received data: {stock_data}")

        # Write the data to the CSV file
        writer.writerow([
            stock_data["symbol"],
            stock_data["timestamp"],
            stock_data["open"],
            stock_data["high"],
            stock_data["low"],
            stock_data["close"],
            stock_data["volume"]
        ])
    ])
```

Figure 31

PART 3

Reflective Report on Part 1 and Part 2

Part 1: NHS England Prescription Data Analysis

Learning Experience

Through this project, I gained a deep understanding of big data processing using Apache Spark. The structured approach to data cleaning, feature engineering, and model building taught me the importance of preparing data thoroughly before applying machine learning models. Working with Power BI also enhanced my skills in data visualisation, enabling me to present complex findings clearly.

Difficulties Encountered

The primary challenge was handling the dataset's skewness and outliers, significantly affecting the model's performance. Additionally, working with null values and categorical variables required advanced feature engineering techniques, which were initially overwhelming.

Overcoming Challenges

To address these difficulties, I relied on research and experimentation. Applying log transformations and scaling techniques helped normalise skewed data while clustering geographical positions provided better insights. Leveraging online resources and documentation on PySpark proved invaluable in resolving technical hurdles.

Part B: Real-Time Stock Data Streaming Using Kafka

Learning Experience

This part of the project introduced me to Kafka's real-time data streaming capabilities. I learned how to implement producer-consumer architectures and process live data streams, which are crucial for real-time analytics. Additionally, working with the Alpha Vantage API to fetch live stock data was an excellent introduction to API integration.

Difficulties Encountered

Configuring Kafka on a local machine was challenging due to its dependencies and setup requirements. Another obstacle was ensuring the data pipeline operated smoothly without interruptions, especially while processing batches for statistical computations.

Overcoming Challenges

I tackled the Kafka configuration issues by referring to community forums and official documentation. Setting up detailed logging mechanisms helped identify and resolve pipeline

interruptions. Breaking the tasks into smaller modules—producer, consumer, and batch processing—simplified the implementation and ensured the overall system's robustness.

Conclusion

This project gave me valuable hands-on experience in batch and real-time data processing. It reinforced my technical skills while emphasising the importance of a systematic approach to problem-solving. These learnings have equipped me to handle similar challenges in future data science projects.

Appendices

Appendix 1. (Pipeline 1)

```
log_target_column = "LOG_ACTUAL_COST"
df_updated = df_updated.withColumn(log_target_column, log(col("ACTUAL_COST") + 1))

#StringIndexer and OneHotEncoder for categorical variables
indexer = StringIndexer(inputCol="BNF_DESCRIPTION",
outputCol="BNF_DESCRIPTION_INDEX")
encoder = OneHotEncoder(inputCol="BNF_DESCRIPTION_INDEX",
outputCol="BNF_DESCRIPTION_OHE")
chem_indexer = StringIndexer(inputCol="CHEMICAL_SUBSTANCE_BNF_DESCR",
outputCol="CHEMICAL_SUBSTANCE_BNF_DESCR_INDEX")

#Log-transform numerical features
log_columns = ["TOTAL_QUANTITY", "NIC", "ADQUSAGE"]
for col_name in log_columns:
    df_updated = df_updated.withColumn(f"LOG_{col_name}", log(col(col_name) + 1))

#Assemble and scale log-transformed features
scaled_features = []
for col_name in log_columns:
    assembler = VectorAssembler(inputCols=[f"LOG_{col_name}"],
outputCol=f"{col_name}_vec")
    scaler = StandardScaler(inputCol=f"{col_name}_vec", outputCol=f"{col_name}_scaled",
withMean=True, withStd=True)
    scaled_features.append(assembler)
    scaled_features.append(scaler)

#MinMax scaling for QUANTITY, YEAR, MONTH
minmax_columns = ["QUANTITY", "YEAR", "MONTH"]
minmax_scalers = []
for column in minmax_columns:
    assembler = VectorAssembler(inputCols=[column], outputCol=f"{column}_vec")
    scaler = MinMaxScaler(inputCol=f"{column}_vec", outputCol=f"{column}_scaled")
    minmax_scalers.append(assembler)
    minmax_scalers.append(scaler)

#KMeans clustering for geolocation
geo_assembler = VectorAssembler(inputCols=["PRACTICE_LATITUDE",
"PRACTICE_LONGITUDE"], outputCol="geo_features")

#VectorAssembler for all features
final_features = [
    "BNF_DESCRIPTION_OHE", "CHEMICAL_SUBSTANCE_BNF_DESCR_INDEX",
    "TOTAL_QUANTITY_scaled",
    "NIC_scaled", "ADQUSAGE_scaled", "QUANTITY_scaled", "YEAR_scaled",
    "MONTH_scaled"
]
```

```

feature_assembler = VectorAssembler(inputCols=final_features, outputCol="features")

#Linear Regression
lr = LinearRegression(featuresCol="features", labelCol=log_target_column)

# Pipeline assembly
stages = [
    indexer, encoder, chem_indexer,
    *scaled_features, *minmax_scalers,
    geo_assembler, feature_assembler, lr
]
log_pipeline = Pipeline(stages=stages)

# Train-test split
train_data, test_data = df_updated.randomSplit([0.8, 0.2], seed=42)

# Fit the pipeline and evaluate the model
log_model = log_pipeline.fit(train_data)
log_predictions = log_model.transform(test_data)

#Evaluate model performance
evaluator_rmse = RegressionEvaluator(labelCol=log_target_column,
predictionCol="prediction", metricName="rmse")
evaluator_r2 = RegressionEvaluator(labelCol=log_target_column,
predictionCol="prediction", metricName="r2")

rmse = evaluator_rmse.evaluate(log_predictions)
r2 = evaluator_r2.evaluate(log_predictions)

print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"R-squared (R2): {r2}")

# Display sample predictions
log_predictions.select("prediction", log_target_column, "features").show(truncate=False)

```

Appendix2:(Pipeline 2)

```

# Linear Regression for ACTUAL_COST
lr_original = LinearRegression(featuresCol="features", labelCol="ACTUAL_COST",
predictionCol="prediction")

# Define the pipeline for original target variable
original_pipeline = Pipeline(stages=stages[:-1] + [lr_original]) # Replace only the
LinearRegression stage

# Fit and evaluate the model
original_model = original_pipeline.fit(train_data)
original_predictions = original_model.transform(test_data)

```

```

evaluator_rmse = RegressionEvaluator(labelCol="ACTUAL_COST",
predictionCol="prediction", metricName="rmse")
evaluator_r2 = RegressionEvaluator(labelCol="ACTUAL_COST",
predictionCol="prediction", metricName="r2")

rmse_original = evaluator_rmse.evaluate(original_predictions)
r2_original = evaluator_r2.evaluate(original_predictions)

print(f"Original Target Variable Metrics:")
print(f"Root Mean Squared Error (RMSE): {rmse_original}")
print(f"R2: {r2_original}")

```

Appendix 3: (Streamlit Dashboard)

```

import streamlit as st
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Define scaling parameters
scaling_params = {
    "TOTAL_QUANTITY": {"mean": 1000.0, "std": 300.0, "min": 0.0, "max": 5000.0},
    "NIC": {"mean": 50.0, "std": 10.0, "min": 10.0, "max": 100.0},
    "ADQUSAGE": {"mean": 200.0, "std": 50.0, "min": 50.0, "max": 500.0},
    "QUANTITY": {"min": 10.0, "max": 200.0},
    "YEAR": {"min": 2020.0, "max": 2023.0},
    "MONTH": {"min": 1.0, "max": 12.0},
}

# Reverse transformation function
def reverse_transform_data(data):
    """
    Reverse transformations for scaled and log-transformed data.
    :param data: Transformed pandas DataFrame
    :return: Original DataFrame
    """
    if "LOG_ACTUAL_COST" in data.columns:
        data["ACTUAL_COST"] = np.exp(data["LOG_ACTUAL_COST"]) - 1

    if "TOTAL_QUANTITY_scaled" in data.columns:
        params = scaling_params["TOTAL_QUANTITY"]
        data["TOTAL_QUANTITY"] = (data["TOTAL_QUANTITY_scaled"] * params["std"])
        + params["mean"]

    if "NIC_scaled" in data.columns:
        params = scaling_params["NIC"]
        data["NIC"] = (data["NIC_scaled"] * params["std"]) + params["mean"]

    if "ADQUSAGE_scaled" in data.columns:
        params = scaling_params["ADQUSAGE"]

```

```

    data["ADQUSAGE"] = (data["ADQUSAGE_scaled"] * params["std"]) +
params["mean"]

    if "QUANTITY_scaled" in data.columns:
        params = scaling_params["QUANTITY"]
        data["QUANTITY"] = (data["QUANTITY_scaled"] * (params["max"] -
params["min"]))) + params["min"]

    if "YEAR_scaled" in data.columns:
        params = scaling_params["YEAR"]
        data["YEAR"] = (data["YEAR_scaled"] * (params["max"] - params["min"]))) +
params["min"]

    if "MONTH_scaled" in data.columns:
        params = scaling_params["MONTH"]
        data["MONTH"] = (data["MONTH_scaled"] * (params["max"] - params["min"]))) +
params["min"]

    return data

# Dashboard function
def dashboard(data):
    st.title("Machine Learning Dashboard")

    st.header("Original and Transformed Data")
    view_original = st.checkbox("View Original Data")

    if view_original:
        original_data = reverse_transform_data(data.copy())
        st.dataframe(original_data)
    else:
        st.dataframe(data)

    st.header("Visualizations")
    st.write("Choose columns to visualize:")
    columns = st.multiselect("Select columns for visualization", data.columns)

    if len(columns) > 1:
        st.line_chart(data[columns])
    elif len(columns) == 1:
        st.bar_chart(data[columns])

    st.header("Search and Filter")
    search_column = st.selectbox("Select a column to search", data.columns)
    search_value = st.text_input("Enter a value to filter")
    if search_value:
        filtered_data = data[data[search_column].astype(str).str.contains(search_value)]
        st.dataframe(filtered_data)

# Main Streamlit app function

```

```

def main():
    # Load the transformed dataset
    uploaded_file = st.file_uploader("Upload your transformed CSV file", type=["csv"])
    if uploaded_file is not None:
        data = pd.read_csv(uploaded_file)
        st.write("Data Loaded Successfully!")
        dashboard(data)
    else:
        st.write("Please upload a CSV file to continue.")

if __name__ == "__main__":
    main()

```

References

Chambers, B. and Zaharia, M. (2018) *Spark: The Definitive Guide*. Sebastopol, CA: O'Reilly Media.

Ahmed, N., Barczak, A.L.C., Susnjak, T., and Rashid, M.A. (2020) 'A Comprehensive Performance Analysis of Apache Hadoop and Apache Spark for Large Scale Data Sets Using HiBench', *Journal of Big Data*, 7(110). Available at: <https://journalofbigdata.springeropen.com/articles/10.1186/s40537-020-00388-5> (Accessed: 6 December 2024).

Dong, X. & Liu, C. (2018) *Feature Engineering for Machine Learning and Data Analytics*. CRC Press.

Géron, E. (2019) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. 2nd edn. O'Reilly Media.

Kuhn, M. & Johnson, K. (2019) *Feature Engineering and Selection: A Practical Approach for Predictive Models*. CRC Press.